

Lockin

SE4-DMAD eksamenskompendium

Diskret matematik + Algoritmer og datastrukturer

Ordnet efter opgavetype. Hvert kapitel giver en genbrugelig opskrift,
derefter de tilbagevendende eksamensopgaver løst — bare indsæt de nye tal.

Rasmus Liltorp

Indhold

Find en opgave efter type	4
Del I — Diskret matematik	10
Udsagnslogik og sandhedstabeller	11
Sådan løser du den	11
Tilbagevendende eksamensspørgsmål	12
Prædikatalogik og kvantorer	15
Sådan løser du den	15
Tilbagevendende eksamensspørgsmål	16
Bevismetoder	19
Sådan løser du den	19
Tilbagevendende eksamensspørgsmål	20
Matematisk induktion	22
Sådan løser du den	22
Tilbagevendende eksamensspørgsmål	23
Relationer, ækvivalens og ordning	25
Sådan løser du den	25
Tilbagevendende eksamensspørgsmål	26
Del II — Algoritmer og datastrukturer	29
Asymptotisk analyse	30
Sådan løser du den	30
Tilbagevendende eksamensspørgsmål	32
Rekursionsligninger og Master Theorem	34
Sådan løser du den	34
Tilbagevendende eksamensspørgsmål	35
Sortering og udvælgelse	39
Sådan løser du den	39
Tilbagevendende eksamensspørgsmål	40
Divide & Conquer	43
Sådan løser du den	43
Tilbagevendende eksamensspørgsmål	44
Dynamisk programmering	47
Sådan løser du den	47
Tilbagevendende eksamensspørgsmål	48
Greedy-algoritmer	51
Sådan løser du den	51
Tilbagevendende eksamensspørgsmål	52
Prioritetskøer og heaps	55
Sådan løser du den	55

Tilbagevendende eksamensspørgsmål	56
Søgestrukturer: BST, augmenteret BST, hashing	59
Sådan løser du den	59
Tilbagevendende eksamensspørgsmål	61
Disjunkte mængder (Union-Find)	64
Sådan løser du den	64
Tilbagevendende eksamensspørgsmål	65
Grafgennemløb og sammenhængskomponenter	68
Sådan løser du den	68
Tilbagevendende eksamensspørgsmål	69
Minimum udspændende træer	72
Sådan løser du den	72
Tilbagevendende eksamensspørgsmål	73
Korteste veje	76
Sådan løser du den	76
Tilbagevendende eksamensspørgsmål	77
Korrekthed og løkkeinvarianter	80
Sådan løser du den	80
Tilbagevendende eksamensspørgsmål	81
Appendiks	84
Appendix: Logaritmer og talrepræsentation	85
Sådan løser du det	85
Tilbagevendende eksamensspørgsmål	86

Find en opgave efter type

Hver gennemregnet eksamensopgave står her, sorteret efter opgavetype. Find den type du sidder fast i, klik på opgaven, og hop ned til løsningen. Du kan også søge efter et ord i opgaveteksten.

Udsagnslogik og sandhedstabeller

- **Udsagnslogik: hvilke udsagn er sande?**
Lad p, q, r være udsagn. Hvilke af nedenstående udsagn er sande? (Et eller flere svar.)
- **Udsagnslogik: hvilke udsagn er sande?**
Lad p, q, r, s være udsagn. Hvilke af følgende sammensatte udsagn er sande? (Et eller flere svar.)
- **Udsagnslogik: hvilke udsagn er sande?**
Lad p, q, r være udsagn. Hvilke sammensatte udsagn er sande? (Et eller flere svar.)
- **Udsagnslogik: find de ækvivalente udsagn**
Hvilke udsagn er ækvivalente med $\neg(p \wedge q)$?

Prædikatalogik og kvantorer

- **Kvantorer: hvilke udsagn er sande?**
Hvilke udsagn er sande? (Et eller flere korrekte svar.)
- **Kvantorer: hvilke udsagn er sande? (+ fornægtelse)**
Domæne $\mathbb{Z}$. Hvilke udsagn er sande? (Et eller flere korrekte svar.)
- **Kvantorer: hvilke udsagn er sande? (+ fornægtelse)**
Domæne $\mathbb{Z}$. Hvilke udsagn er sande? (Et eller flere korrekte svar.)
- **Kvantorer: sandhedsværdi + fornægt uden $\neg$**
 $\mathbb{N} = \{0, 1, 2, \dots\}$. Først (4a): hvilke af i og ii er sande?
 1. $\forall x \in \mathbb{N} : \exists y \in \mathbb{N} : x < y$
 2. $\exists y \in \mathbb{N} : \forall x \in \mathbb{N} : x < y$Dernæst (4b): angiv fornægtelsen af i uden $\neg$ i svaret.

Bevismetoder

- **Bevismetode: direkte bevis om lige/ulige**
Brug et direkte bevis til at vise, at summen af to ulige heltal er lige.
- **Bevismetode: kontraposition og modstrid**
Vis, at hvis n er et heltal og $n^3 + 5$ er ulige, så er n lige — ved (a) kontraposition, (b) modstrid.
- **Bevismetode: modstrid om rationalt/irrationelt**
Bevis eller modbevis: produktet af et rationalt tal forskelligt fra nul og et irrationelt tal er irrationelt.
- **Bevismetode: vælg det gyldige bevis**
Angiv et korrekt bevis for: $a + b > c \rightarrow a > \frac{c}{2} \vee b > \frac{c}{2}$.

Matematisk induktion

- **Induktion: hvilke kandidatbeviser er gyldige?**
Bevis at $3^n - 1$ er lige for alle $n \in \mathbb{N}$. Hvilke argumenter er gyldige induktionsbeviser? (en eller flere korrekte)
- **Induktion: hvilke kandidatbeviser er gyldige?**
Påstand: $2^n + 3^n < 4^n$ for $n \geq 2$. Hvilke af 5.a–5.g er gyldige induktionsbeviser?
- **Induktion: hvilke kandidatbeviser er gyldige?**
Hvilke er korrekte induktionsbeviser for $\sum_{i=1}^n \left(\left(\frac{1}{2} \right)^i - \left(\frac{1}{2} \right)^{i+1} \right) = 1 - \left(\frac{1}{2} \right)^{n+1}$ for alle $n \geq 1$? (en eller flere korrekte)

Relationer, ækvivalens og ordning

- **Relation: hvilke egenskaber har den?**
Lad $R = \{(a, a), (a, b), (b, a), (c, c)\}$ være en relation på $\{a, b, c\}$. Hvilke udsagn er sande?
- **Lukninger: er den angivne lukning korrekt?**
Lad $A = \{a, b, c, d\}$. Hvilke udsagn er sande?
- **Lukning: udregn den transitive lukning**
Betragt relationen på $\{a, b, c\}$: $R = \{(a, b), (b, a), (b, b), (b, c)\}$. Angiv den transitive lukning af R .

► **Lukning: udregn den symmetriske lukning**

Angiv den symmetriske lukning af $T = \{(a, b), (b, b), (c, d), (d, e)\}$.

► **Relation: hvilke egenskaber har den? (digraf)**

En digraf for relationen S på $\{a, b, c, d, e, f\}$ har løkker på a, b, e, f , kanterne $a \rightarrow b, a \rightarrow c, a \rightarrow d, a \rightarrow e, b \rightarrow e, c \rightarrow d, f \rightarrow d$, og tovejsparret $e \leftrightarrow f$. Hvilke af følgende gælder: refleksiv, symmetrisk, antisymmetrisk, transitiv, ækvivalensrelation, partiel ordning, total ordning?

► **Relation: hvilke er ækvivalensrelationer?**

På $\{a, b, c\}$, hvilke af disse er ækvivalensrelationer?

Asymptotisk analyse

► **O-notation: er $X = O(Y)$?**

Hvilke af følgende er sande? (Et eller flere svar.)

► **Asymptotik: $O / \Omega / \Theta / o / \omega$ sand?**

Hvilke af følgende er sande? (Et eller flere svar.)

► **Køretid: tæl løkkernes gennemløb**

Hvad er den asymptotiske køretid i Θ -notation?

```
ALGORITME3(n): i = 1; while i <= n: { j = n; while j > 1: j = j - 1; i = 2*i }
```

► **Køretid: løkke-fælde (tæller nulstilles ikke)**

Hvad er den asymptotiske køretid i Θ -notation?

```
ALGORITME2(n): i = 1; j = 1; while i <= n: { i = i + 5; while j < i: j = j + 1 }
```

Rekursionsligninger og Master Theorem

► **Master Theorem: løs rekursionsligning**

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = 2 \cdot T(n/4) + n^2$

► **Master Theorem: løs rekursionsligning**

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = 4 \cdot T(n/2) + n^2$

► **Master Theorem: løs rekursionsligning**

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = 4 \cdot T(n/3) + n$

► **Master Theorem: løs rekursionsligning**

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = T(n/4) + 1$

► **Master Theorem: kan den løses?**

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = 5 \cdot T(n/5) + n \log n$

Sortering og udvælgelse

► **Sortering: værste-falds-køretid for CountingSort**

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^3]$. Hvad er værste-falds-køretiden for CountingSort på dette input?

► **Sortering: værste-falds-køretid for RadixSort**

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^3]$. Hvad er værste-falds-køretiden for RadixSort, når heltallene behandles som tre cifre med værdier i intervallet $[0, n)$?

► **Sortering: værste-falds-køretid for QuickSort**

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^3]$. Hvad er værste-falds-køretiden for QuickSort på dette input?

► **Sortering: hvilke er $\Theta(n^2)$ i værste fald?**

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^2]$. Ved TreeSort menes algoritmen, der indsætter tallene ét ad gangen i et søgetræ og derefter laver et inorder-gennemløb. Hvilke af algoritmerne nedenfor har værste-falds-køretid $\Theta(n^2)$? (Et eller flere svar.)

► **QuickSort: kør PARTITION i hånden**

Kør PARTITION($A, 1, 7$) på arrayet $A = [6, 2, 4, 5, 1, 7, 3]$ (indeks 1..7). Hvilken mulighed viser A bagefter? (Standard CLRS Lomuto-partition; pivot er sidste element $A[7] = 3$.)

► **CountingSort: trace tælle-arrayet C**

Et array af ni heltal med værdier 0..6: $A = [2, 0, 6, 2, 3, 5, 5, 1, 2]$ (indeks 1..9). Kør COUNTING-SORT($A, 9, 6$) med et array C med syv pladser (0..6). Hvad er summen af de syv heltal i C ved terminering?

Divide & Conquer

► **Strassen: køretid for naiv blokmultiplikation**

Naiv rekursiv matrixmultiplikation deler hver $n \times n$ -matrix i fire blokke og kalder rekursivt med 8 delmultiplikationer:

$$T(n) = 8T(n/2) + n^2$$

Hvilken Θ -grænse gælder, og slår den den naive $\Theta(n^3)$?

► **Strassen: køretid med 7 delprodukter**

Strassen regner de fire outputblokke ud med 7 delmultiplikationer i stedet for 8:

$$T(n) = 7T(n/2) + n^2$$

Hvad er køretiden?

► **Max-sum: køretid og plads for Kadane**

Hvad er køretid og ekstra pladsforbrug for **Kadanes algoritme (MaxSum3)** til maximum subarray-problemet?

► **Master Theorem: løs rekursionsligning**

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = 1T(n/2) + n^{1/2}$$

Dynamisk programmering

► **DP: aflæs køretid af given rekursion**

For et optimeringsproblem er inputtet en omkostning c_{ij} for alle i og j . En løsning $b(i, j)$ kan beskrives ved rekursionen

$$b(i, j) = \begin{cases} 0 & \text{hvis } i = 0 \text{ eller } j = 0 \\ c_{ij} + \min\{b(i-1, k) : 0 \leq k < j\} & \text{hvis } i > 0 \text{ og } j > 0 \end{cases}$$

Hvis $b(m, n)$ findes ved dynamisk programmering, hvilken køretid opnås da?

► **DP: mindste pladsforbrug for given rekursion**

Samme problem og samme rekursion for $b(i, j)$. Hvis $b(m, n)$ findes ved dynamisk programmering, hvad er det mindste pladsforbrug, der kan opnås?

► **DP: aflæs køretid af given rekursion**

Find $B(n)$, antallet af binære træer med n knuder (fx $B(3) = 5$), ved rekursionen

$$B(0) = 1, \quad B(n) = \sum_{i=0}^{n-1} B(i) \cdot B(n-i-1) \text{ for } n > 0$$

Beregnet ved dynamisk programmering på denne rekursion, hvilken køretid opnås?

► **DP: aflæs køretid af given rekursion (trekant)**

For et optimeringsproblem er inputtet værdier c_k for $k = 1, 2, \dots, n$. En løsning $L(i, j)$ kan beskrives ved rekursionen

$$L(i, j) = \begin{cases} c_i & \text{hvis } i = j \\ i \cdot L(i+1, j) + j \cdot L(i, j-1) & \text{hvis } i < j \end{cases}$$

Hvis $L(1, n)$ findes ved dynamisk programmering, hvilken køretid opnås?

Greedy-algoritmer

► **Huffman: kodeordslængde for et symbol**

En fil indeholder tegn med hyppigheder: o=150, p=100, q=25, r=125, s=200, t=50, u=75. Byg et Huffman-træ. Hvor mange bit er der i kodeordet for q ?

► **Huffman: samlet antal bit**

Et Huffman-træ for en fil med hyppigheder o=150, p=100, q=25, r=125, s=200, t=50, u=75 (i alt 725 symboler). Hvor mange bit fylder de 725 symboler tilsammen Huffman-kodet?

► **Huffman: kodeordslængde for et symbol**

En fil indeholder tegnene med hyppigheder: a=200, b=250, c=100, d=350, e=400. Byg et Huffman-træ. Hvor mange bit er der i kodeordet for d ?

► **Huffman: hvilke optimale træer er producerbare?**

Frekvenser a=100, b=150, c=150, d=250, e=350 (i alt 1000). Alle de viste træer er optimale (pris 2250 bit). Hvilke af træerne H2–H5 kan Huffman faktisk producere?

Prioritetskøer og heaps

► Heap: er arrayet en (min-)heap?

Hvilke af følgende arrays af størrelse 10 er min-heaps?

$A_1 = [7, 4, 9, 2, 6, 8, 10, 1, 3, 5]$

$A_2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

$A_3 = [1, 2, 3, 4, 1, 2, 3, 4, 5, 6]$

$A_4 = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]$

► Heap: trace Extract-Min skridt for skridt

Udfør Extract-Min på min-heapen $A = [3, 5, 6, 10, 11, 8, 7, 18, 16, 15]$ (1-indeksret, positioner 1..10). Hvilken position i A ender 15 på bagefter?

► Heap: trace Increase-Key + Extract-Max

Udfør først Heap-Increase-Key($A, 9, 15$) og derefter Heap-Extract-Max(A) på max-heapen A herunder. A (positioner 1..9) = $[18, 9, 16, 4, 8, 12, 13, 1, 2]$. Hvilket af svarene viser heapen efter de to operationer?

► Heapsort: køretid på ens elementer

Hvad er worst-case køretiden for Heapsort, når den køres på n identiske elementer?

Søgestrukturer: BST, augmented BST, hashing

► Augmenteret BST: vælg opdateringsligninger

En mængde punkter i planen gemmes i et balanceret binært søgetræ (BST) med punkternes x -koordinat som nøgle. Hver knude svarer til et punkt og gemmer derudover fire felter for hele sit deltræ: $v.xmax, v.xmin, v.ymax, v.ymin$. Hvilket sæt opdateringsligninger vedligeholder felterne korrekt i knude v ud fra dens børn $v.left, v.right$ og dens egne koordinater $v.x, v.y$?

► Augmenteret BST: hvorfor bevares $O(\log n)$?

Lad nu søgetræet være et rød-sort træ. Hvilke af følgende er korrekte argumenter for, at informationen i træets knuder kan vedligeholdes under indsættelser og sletninger, uden at ændre de rød-sortede træers køretid på $O(\log n)$ for indsættelser og sletninger?

► Hashing: hvilke h' -værdier passer? (linear probing)

En hashtabel H bruger linear probing og en hjælpe-hashfunktion $h'(x)$. Tabellen er nu (indeks 0..6): plads 0 = 12, plads 1 tom, plads 2 = 10, plads 3 tom, plads 4 = 22, plads 5 tom, plads 6 = 31. Derefter indsættes 97, og tabellen er bagefter: plads 0 = 12, plads 1 = 97, plads 2 = 10, plads 3 tom, plads 4 = 22, plads 5 tom, plads 6 = 31. Tabelstørrelse $m = 7$. Hvilke værdier af $h'(97)$ er mulige? (et eller flere svar)

► Sortering: hvilke er $\Theta(n^2)$ i værste fald?

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^2]$. Med TreeSort menes algoritmen, der indsætter tallene ét ad gangen i et søgetræ og derefter laver et inorder-gennemløb. Hvilke af følgende algoritmer har worst-case-køretid $\Theta(n^2)$? (et eller flere svar)

Disjunkte mængder (Union-Find)

► Union-Find: tegn skoven efter operationerne

Disjunkt-mængde-skov med union by rank og path compression (Cormen 4. udg.). Fra tom: Make-Set på a, b, c, d, e, f , derefter Union(f, e), Union(b, f), Union(d, a), Union(f, d), Union(b, c), Find-Set(a). Hvilken figur viser strukturen bagefter?

► Union-Find: tegn skoven (med/uden komprimering)

Kør instruktionerne Union(b, a), Union(b, c), Union(e, d), Union(e, c), Union(g, f), Union(e, g) med union by rank. Ved uafgjort hænges root(x) under root(y), og root(y)'s rank forhøjes. Tegn skoven (b) uden path compression og (c) med.

► Union-Find i Kruskal: næste kant + skoven

De første 5 Kruskal-kanter er valgt, så grupperne er $\{E, F, H, I\}, \{D, G\}, \{A, B\}, \{C\}$. Hvilken kant tilføjer Kruskal som den 6., og hvordan ser disjunkt-mængde-skoven ud bagefter, med union by rank + path compression?

Grafgennemløb og sammenhængskomponenter

► BFS: kørs i hånden, find afstand/rækkefølge

Udfør BFS(G, a) med start i knude a og nabolister sorteret alfabetisk. Orienterede kanter: $f \rightarrow g, g \rightarrow j, i \rightarrow j, c \rightarrow f, c \rightarrow g, d \rightarrow g, d \rightarrow i, h \rightarrow c, b \rightarrow c, e \rightarrow b, e \rightarrow j, a \rightarrow d, a \rightarrow e$. Hvilken knude er den første, der får tildelt d -værdi (afstand) 4?

► DFS: klassificér kanter (tæl forward/back/cross)

DFS-Visit(G, a) på den orienterede graf med kanter $a \rightarrow b, b \rightarrow c, c \rightarrow d, a \rightarrow e, b \rightarrow f, c \rightarrow f, c \rightarrow g, f \rightarrow a, f \rightarrow g, d \rightarrow h, h \rightarrow g$. Hvor mange forward-kanter er der i dette gennemløb?

► **Topologisk sortering: er rækkefølgen gyldig?**

Orienteret graf med kanter $a \rightarrow b, a \rightarrow d, b \rightarrow c, b \rightarrow d, c \rightarrow e, c \rightarrow f, e \rightarrow f, g \rightarrow c, g \rightarrow f$. Hvilke lister er en topologisk sortering? (Et eller flere svar.)

► **DFS: kø i hånden, find opdagelses-/afslutningstid**

Udfør DFS med start i i med nabolisterne sorteret alfabetisk. Hvilken knude får opdagelsestid (starttid) 12? Hjulgraf med center i og ydre knuder a til h . Eger: $i \rightarrow a, i \rightarrow b, c \rightarrow i, i \rightarrow d, e \rightarrow i, f \rightarrow i, i \rightarrow g, i \rightarrow h$. Ydre kanter: $h \rightarrow a, b \rightarrow a, b \rightarrow c, d \rightarrow c, e \rightarrow d, f \rightarrow e, g \rightarrow f, g \rightarrow h$.

Minimum udspændende træer► **Kruskal: hvilken kant tilføjes sidst?**

Brug Kruskals algoritme til at finde et MST (læs næste spørgsmål først). Knuder $a, b, c, e, f, g, h, i, j$. Kanter med vægte: $(a, c) = 2, (a, b) = 15, (b, e) = 5, (b, f) = 8, (c, e) = 20, (c, j) = 17, (e, h) = 11, (e, j) = 9, (f, h) = 6, (f, g) = 3, (g, i) = 19, (h, i) = 16, (h, j) = 21, (i, j) = 14$. Hvilken kant tilføjes sidst til MST'et?

► **Kruskal: antal komponenter ved første forkastning**

Fortsæt med Kruskals algoritme på samme graf. I det første øjeblik hvor en undersøgt kant ikke tages med, hvor mange sammenhængskomponenter har (V, A) ? (V er alle knuder, A er de kanter der er taget indtil nu.)

► **Prim: hvilken knude tilføjes sidst?**

Brug Prim's algoritme til at finde et MST med start i knude a . Urettede vægtede kanter: $bc = 5, ca = 11, af = 10, fe = 7, bd = 13, ch = 9, ah = 2, ai = 3, fi = 4, eg = 8, dh = 1, hi = 6, ig = 12$. Hvilken knude tilføjes sidst til MST'et?

► **Kruskal: hvilken kant forkastes først?**

Kør Kruskals algoritme på grafen G_3 . Knuder $a, b, c, d, e, f, g, h, i$ med vægtede kanter: $(a, f) = 3, (a, b) = 4, (f, g) = 8, (f, b) = 5, (g, b) = 2, (g, c) = 6, (b, c) = 1, (h, c) = 9, (h, d) = 6, (h, i) = 9, (i, d) = 7, (i, e) = 1, (c, d) = 8, (d, e) = 7$. Hvilken kant er den første der undersøges af algoritmen, men ikke tages med i MST'et?

Korteste veje► **Korteste veje: vælg algoritme til grafen**

Which of the following algorithms can be used to find shortest paths on this graph? (Et eller flere svar.) G_1 er orienteret med knuder a, b, c, d, e og alle vægte 1, og grafen har kredse: $b \rightarrow a, a \rightarrow d, a \rightarrow e, b \rightarrow e, e \rightarrow d, c \rightarrow b, c \rightarrow e, c \rightarrow d$.

► **Korteste veje: vælg algoritme (DAG, negative vægte)**

Which of the following algorithms can be used to find shortest paths on this graph? (Et eller flere svar.) G_2 er orienteret med knuder a, b, c, d, e , nogle vægte -1 , og grafen er en DAG (topologisk orden c, b, a, e, d): $b \rightarrow a(1), a \rightarrow d(-1), a \rightarrow e(1), b \rightarrow e(-1), e \rightarrow d(1), c \rightarrow b(1), c \rightarrow e(-1), c \rightarrow d(1)$.

► **Dijkstra: udtrækningsrækkefølge i hånden**

Run Dijkstra's algorithm on graph G_2 below, starting at node a . The first node extracted from the priority queue (via EXTRACT-MIN) during the run is node a . Which node is the sixth one extracted? (Ved uafgjort: alfabetisk mindste navn.) Kanter: $a \rightarrow b(1), a \rightarrow d(3), d \rightarrow g(1), d \rightarrow e(3), g \rightarrow h(3), g \rightarrow e(1), h \rightarrow i(1), b \rightarrow d(1), b \rightarrow e(3), b \rightarrow c(2), e \rightarrow h(1), e \rightarrow f(2), c \rightarrow e(1), c \rightarrow f(3), f \rightarrow h(2), f \rightarrow i(3)$.

► **Bellman-Ford: find alle afstande i hånden**

Kør Bellman-Ford fra a på den orienterede graf G_1 og angiv den endelige $v.d$ for alle knuder. Kanter: $a \rightarrow e(8), a \rightarrow f(10), a \rightarrow b(17), e \rightarrow h(-4), f \rightarrow h(-10), f \rightarrow g(25), g \rightarrow h(-12), g \rightarrow c(-3), b \rightarrow g(-5), c \rightarrow b(19), c \rightarrow d(2), d \rightarrow e(6), h \rightarrow d(1)$.

Korrekthed og løkkeinvarianter► **Løkkeinvariant: bevis invariant + aflæs output**

Betragt $\text{KvadratRod}(n)$:

```

 $i \leftarrow 0; \quad s \leftarrow 0$ 
while  $s \leq n$ :  $s \leftarrow s + 2i + 1; \quad i \leftarrow i + 1$ 
 $r \leftarrow i - 1$ ; return  $r$ 

```

(a) Bevis løkkeinvarianten: ved starten af while-løkken (altså ved løkketesten) gælder $s = i^2$.

(b) Brug invarianten til at vise, at KvadratRod returnerer det største heltal $\leq \sqrt{n}$, altså $r = \lfloor \sqrt{n} \rfloor$.

► **Løkkeinvariant: hvilke udsagn er invarianter?**

Betragt `Factorial(n)`:

```

i ← n;  r ← 1
while i > 1:  r ← r · i;  i ← i - 1
return r

```

For heltal $n \geq 1$, angiv for hvert udsagn om det er en løkkeinvariant (sandt hver gang while-testen evalueres).

► **Løkkeinvariant: bevis invariant + aflæs output**

`IntegerLog(n)` beregner $\lfloor \log n \rfloor$:

```

k ← 0;  i ← n
while i > 1:  i_lige : i ← i/2, k ← k + 1;  ellers : i ← i - 1
return k

```

Brug fakta $\lfloor \log(n/2) \rfloor = \lfloor \log n \rfloor - 1$ og $\lfloor \log(n-1) \rfloor = \lfloor \log n \rfloor$ for ulige n .

(b) Bevis invarianten $\lfloor \log i \rfloor + k = \lfloor \log n \rfloor$.

(c) Vis, at løkken returnerer $\lfloor \log n \rfloor$.

► **Løkkeinvariant: hvilke udsagn er invarianter?**

`LogBaseTo(n)` beregner $\lceil \log_2 n \rceil$:

```

x ← 1;  r ← 0
while x < n:  x ← 2x;  r ← r + 1
return r

```

Hvilke af udsagnene nedenfor er en løkkeinvariant for algoritmen (altid sand når testen ved starten af while-løkken evalueres) for alle heltal $n \geq 1$? (Et eller flere svar.)

Appendix: Logaritmer og talrepræsentation

► **O-notation: er $X = O(Y)$?**

Hvilke udsagn er sande? $\log n$ er base to. (Et eller flere svar.)

► **Asymptotik: $O / \Omega / \Theta / o / \omega$ sand?**

Hvilke udsagn er sande? $\log n$ er base to. (Et eller flere svar.)

► **Talrepræsentation: omregn til binær**

Omregn $N = 25$ til binær.

► **Køretid: tæl løkkernes gennemløb**

Algoritmen finder de binære cifre b_i for et positivt heltal n (her $n = 55$). Identiteten $x = y \cdot (x \text{ div } y) + (x \text{ mod } y)$ gælder for alle heltal. Hvad er køretiden?

```

BinaryDigits(n)
  i = 0
  d = n
  while d > 0
    b_i = d mod 2
    d = d div 2
    i = i + 1
  k = i - 1

```

I alt 79 opgaver.

Del I — Diskret matematik

Udsagnslogik og sandhedstabeller

Et udsagn er enten sandt eller falsk. Konnektiverne $\neg$ (ikke), $\wedge$ (og), $\vee$ (eller), $\rightarrow$ (medfører), $\leftrightarrow$ (hvis og kun hvis) og $\oplus$ (XOR) binder udsagn sammen til større udsagn.

En sandhedstabel skriver alle muligheder op. Hver variabel er sand (T) eller falsk (F), så n variable giver

$$2^n$$

rækker. For hver række afgør du, om hele udsagnet er sandt.

Eksamen giver en liste påstande og spørger, hvilke der er sande. Du skal kunne fire ting: afgøre om to udsagn er ækvivalente, om et udsagn er en tautologi, om det er opfyldeligt, og antal rækker i tabellen.

Sådan løser du den

Lær de fire grundkonnektiver udenad.

p	q	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
T	T	T	T	T	T
T	F	F	T	F	F
F	T	F	T	T	F
F	F	F	F	T	T

Konnektivernes sandhed

$p \rightarrow q$ er kun falsk i rækken $T \rightarrow F$; er p falsk, er implikationen automatisk sand. $p \leftrightarrow q$ er sand når p og q er ens; $p \oplus q$ når de er forskellige.

Byg en sandhedstabel

1. Tæl de forskellige variable. Med n får tabellen 2^n rækker, én per kombination af T og F.
2. Lav en kolonne per deludtryk, inderste først: negationer, så $\wedge$ og $\vee$, så $\rightarrow$, $\leftrightarrow$ og $\oplus$.
3. Udfyld kolonnerne efter tabellen ovenfor, op til hele formen.
4. Læs sidste kolonne: alle T er tautologi, alle F kontradiktion, blandet kontingens.

Svar på de fire spørgsmålstyper

1. **Ækvivalens** ($A \equiv B$): stil de to kolonner op side om side. Ens i alle rækker betyder ækvivalente; én afvigende række afgør det.
2. **Tautologi**: sidste kolonne er T overalt. Genvej for $X \rightarrow Y$: antag det falsk (X sand, Y falsk) og søg en modstrid. Findes ingen, er det en tautologi.
3. **Opfyldelig** ("kan p tildeles, så Z bliver sand"): ja, hvis blot én række gør Z sand.
4. **Antal rækker**: 2^n , hvor n er antal **forskellige** variable.

Lær de tilbagevendende ækvivalenser udenad. De Morgan:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q \quad \neg(p \vee q) \equiv \neg p \wedge \neg q$$

Materiel implikation og kontraposition:

$$p \rightarrow q \equiv \neg p \vee q \quad p \rightarrow q \equiv \neg q \rightarrow \neg p$$

Biimplikation:

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

Tæl forskellige variable

Tæl **forskellige** variable. $p \vee \neg p$ har én, altså 2 rækker, ikke 4. $(p \vee \neg q) \wedge q$ har to, altså 4, ikke 8.

Omvendt og invers

Kontrapositionen $\neg q \rightarrow \neg p$ er ækvivalent med $p \rightarrow q$. Den omvendte $q \rightarrow p$ og den inverse $\neg p \rightarrow \neg q$ er det ikke, men er ækvivalente med hinanden.

Tilbagevendende eksamensspørgsmål

MCQ juni 2023, Spm. 33

Lad p, q, r være udsagn. Hvilke af nedenstående udsagn er sande? (Et eller flere svar.)

- (a) $(p \wedge q) \vee (p \wedge \neg q)$ er ækvivalent med p .
- (b) Hvis $p \leftrightarrow \neg q$ er sand, så er $p \wedge \neg q$ også sand.
- (c) $p \rightarrow (p \vee q)$ er en tautologi.
- (d) Hvis q er sand og r er falsk, kan p tildeles en sandhedsværdi, så $(\neg p \wedge q) \vee (p \wedge r)$ er sand.
- (e) Sandhedstabellen for $(p \rightarrow q) \wedge r$ har otte rækker.

Svar. Sande: 0, 2, 3, 4. Falsk: 1.

Fremgangsmåde. (0) $(p \wedge q) \vee (p \wedge \neg q) = p \wedge (q \vee \neg q) = p$. Sand.

- (1) $p = F, q = T$: $p \leftrightarrow \neg q$ bliver $F \leftrightarrow F = T$, men $p \wedge \neg q = F$. Falsk.
- (2) Sand om p er sand ($p \vee q$ sand) eller falsk (tom opfyldelse). Tautologi.
- (3) $q = T, r = F$ reducerer formelen til $\neg p$; vælg $p = F$. Opfyldelig.
- (4) Tre variable giver $2^3 = 8$ rækker. Sand.

MCQ juni 2021, Spm. 34

Lad p, q, r, s være udsagn. Hvilke af følgende sammensatte udsagn er sande? (Et eller flere svar.)

- (a) Hvis p er sand og q er falsk, så er $p \vee q \vee r$ sand.
- (b) $p \wedge q$ er falsk hvis og kun hvis $p \vee q$ er falsk.
- (c) Hvis $p \wedge \neg q$ er sand, så er $p \vee \neg q$ også sand.
- (d) $\neg p \vee \neg q$ er ækvivalent med $\neg(p \wedge q)$.
- (e) Hvis $p \leftrightarrow q$ er sand, så er $\neg p \rightarrow \neg q$ også sand.
- (f) Hvis $p \rightarrow q$ er sand og p er falsk, så er q også falsk.
- (g) Sandhedstabellen for $p \vee q \leftrightarrow r \wedge s$ har 8 rækker.
- (h) Sandhedstabellen for $p \vee \neg p$ har 4 rækker.
- (i) p kan tildeles en sandhedsværdi, så $(p \leftrightarrow q) \wedge (p \oplus q)$ er sand.
- (j) Hvis p er sand, kan q tildeles en sandhedsværdi, så $p \wedge (\neg p \vee \neg q)$ er sand.

Svar. Sande: 0, 2, 3, 4, 9. Falske: 1, 5, 6, 7, 8.

Fremgangsmåde. (0) p sand gør disjunktionen sand. Sand.

- (1) $p \wedge q$ er falsk i 3 rækker, $p \vee q$ kun i 1. Falsk.
- (2) $p \wedge \neg q$ tvinger p sand, så $p \vee \neg q$ er sand. Sand.
- (3) De Morgan, præcis ens. Sand.
- (4) $p \leftrightarrow q$ betyder $p = q$, så $\neg p \rightarrow \neg q$ gælder altid. Sand.
- (5) p falsk gør $p \rightarrow q$ sand for begge værdier af q . Falsk.
- (6) Fire forskellige variable giver $2^4 = 16$ rækker, ikke 8. Falsk.
- (7) Én variabel giver $2^1 = 2$ rækker, ikke 4. Falsk.
- (8) $\leftrightarrow$ og $\oplus$ er hinandens negation, så konjunktionen er aldrig sand. Falsk.
- (9) $p = T, q = F$ giver $T \wedge (F \vee T) = T$. Sand.

MCQ juni 2025, Spm. 32

Lad p, q, r være udsagn. Hvilke sammensatte udsagn er sande? (Et eller flere svar.)

- (a) Hvis p er sand og q er falsk, så er $p \vee \neg q$ sand.
- (b) Hvis p er sand, så er $p \oplus p$ også sand.
- (c) Sandhedstabellen for $p \oplus q$ har to rækker.
- (d) Sandhedstabellen for $(p \vee \neg q) \wedge q$ har 8 rækker.
- (e) $\neg(p \wedge q)$ og $\neg p \wedge \neg q$ er ækvivalente.
- (f) $p \rightarrow q$ og $p \vee \neg q$ er ækvivalente.
- (g) $(p \rightarrow q) \rightarrow p$ er en tautologi.
- (h) $p \wedge q$ er en kontingens.

Svar. Sande: 0, 7. Falske: 1, 2, 3, 4, 5, 6.

Fremgangsmåde. (0) $p = T, q = F$ giver $\neg q = T$, så $p \vee T = T$. Sand.

- (1) $p \oplus p = F$ altid. Falsk.
- (2) $p \oplus q$ har to variable, altså $2^2 = 4$ rækker. Falsk.
- (3) $(p \vee \neg q) \wedge q$ har to variable, altså 4 rækker. Falsk.
- (4) Ved $p = T, q = F$: $\neg(T \wedge F) = T$, men $\neg T \wedge \neg F = F$. Den rigtige lov er $\neg(p \wedge q) \equiv \neg p \vee \neg q$. Falsk.
- (5) $p \rightarrow q \equiv \neg p \vee q$, ikke $p \vee \neg q$. Ved $p = F, q = T$ er de forskellige. Falsk.
- (6) Ved $p = F, q = F$: $p \rightarrow q = T$, så $T \rightarrow F = F$. Falsk.
- (7) $p \wedge q$ er kun sand ved (T, T) , falsk ellers. Blandet, altså kontingens. Sand.

DM547 Reeksamen marts 2019, Spm. 3

Hvilke udsagn er ækvivalente med $\neg(p \wedge q)$?

- (a) $p \vee q$
- (b) $\neg p \vee q$
- (c) $\neg p \vee \neg q$
- (d) $(p \oplus q) \vee (\neg p \wedge \neg q)$
- (e) $p \rightarrow q$
- (f) $p \rightarrow \neg q$

(g) $q \rightarrow \neg p$

(h) $p \leftrightarrow \neg q$

Svar. Ækvivalente: 2, 3, 5, 6 (svarende til opg. 3.3, 3.4, 3.6, 3.7).

Fremgangsmåde. Målkolonnen $\neg(p \wedge q)$ er falsk i præcis én række, (T, T) , og sand i de tre andre. Sammenlign hver kandidats kolonne med den.

(0) $p \vee q$ er sand ved (T, T) . Nej.

(1) $\neg p \vee q$ er falsk ved (T, F) . Nej.

(2) De Morgan, præcis ens. Ja.

(3) Sand undtagen ved (T, T) . Ja.

(4) $p \rightarrow q$ er falsk ved (T, F) . Nej.

(5) $p \rightarrow \neg q \equiv \neg p \vee \neg q$. Ja.

(6) $q \rightarrow \neg p \equiv \neg q \vee \neg p$. Ja.

(7) $p \leftrightarrow \neg q$ er falsk ved (T, T) og (F, F) . Nej.

Prædikatalogik og kvantorer

Et prædikat er et udsagn med en fri variabel, fx “ x er lige”. Det har ingen sandhedsværdi, før du binder x . En kvantor binder variabelen og gør prædikatet til et færdigt udsagn.

De to kvantorer er $\forall$ (“for alle”) og $\exists$ (“der findes mindst ét”). De læses altid over et domæne — mængden x kommer fra, typisk $\mathbb{Z}$, $\mathbb{N}$ eller $\mathbb{R}$.

$$\forall x : P(x) \quad \exists x : P(x)$$

Domænet afgør alt: samme prædikat kan skifte sandhedsværdi, når domænet ændres.

Eksamen spørger om tre ting: sandhedsværdien af et (ofte indlejret) udsagn over et givet domæne; fornægtelsen af et udsagn uden $\neg$ tilbage; og rækkefølgen i en kvantorække. I MCQ-versionerne vurderes hvert valg sand/falsk for sig, og flere kan være rigtige.

Sådan løser du den

Sandhedsværdi af et kvantificeret udsagn

1. Skriv domænet op først ($\mathbb{Z}$, $\mathbb{N}$, $\mathbb{R}$, $\mathbb{Z}^+$). Et udsagn kan være sandt over $\mathbb{R}$ og falsk over $\mathbb{Z}$.
2. Læs kvantorrækken udefra og ind, venstre mod højre. Den yderste vælges først; de indre må afhænge af den.
3. Yderste $\forall x$: ét modeksempel gør hele udsagnet falsk.
4. Yderste $\exists x$: angiv ét x , der virker.
5. Indlejret $\forall x \exists y$: hold x fast og byg y som formel i x , fx $y = x + 1$ eller $y = -x$. Virker formelen altid, er udsagnet sandt.
6. Indlejret $\exists x \forall y$: find ét fast x , der får det indre $\forall y$ til at holde for alle y . Test grænsetilfælde som $y = 0$. Ubegrænsede domæner slår som regel disse ihjel.

Rækkefølgen er ikke ligegyldig. $\forall x \exists y$ lader y afhænge af x — hvert x vælger sit eget y . $\exists y \forall x$ kræver ét y , der virker for alle x .

$$\forall x \exists y : P(x, y) \neq \exists y \forall x : P(x, y)$$

Kvantorrækkefølge

Bytter du en $\forall$ og en $\exists$, vender sandhedsværdien næsten altid. Det er den hyppigst testede fælde. “Hvert tal har et større” er sandt over $\mathbb{N}$; “der findes ét tal større end alle” er falsk.

Fornægt et kvantificeret udsagn (intet $\neg$ i svaret)

1. Skub $\neg$ indad én kvantor ad gangen. Hver kvantor vender.
2. Når $\neg$ er inde ved prædikatet, byt relationen ud med sin modsætning uden at efterlade et $\neg$.
 $\neg(a < b)$ bliver $a \geq b$ $\neg(a = b)$ bliver $a \neq b$.
3. Tjek: fornægtelsen skal have modsat sandhedsværdi af originalen.

Vending-reglerne er:

$$\neg \forall x : P(x) \equiv \exists x : \neg P(x)$$

$$\neg \exists x : P(x) \equiv \forall x : \neg P(x)$$

Vend alle kvantorer

De Morgan for kvantorer vender alle kvantorer i rækken. Fornægtelsen af $\exists x \exists y : P$ er $\forall x \forall y : \neg P$, ikke $\forall x \exists y : \neg P$. En “ækvivalens” med kun den ene kvantor vendt er et klassisk falsk MCQ-svar (juni 2021 Spm. 35, juni 2023 Spm. 34).

Vakuøst sand implikation

En implikation inde i en kvantor er vakuøst sand, når forudsætningen er falsk. $\forall x : (x \neq 0 \rightarrow \dots)$ er automatisk opfyldt ved $x = 0$, så $x = 0$ kan aldrig være modeksempel her.

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 33 (flere rigtige)

Hvilke udsagn er sande? (Et eller flere korrekte svar.)

- (a) $\forall x \in \mathbb{Z}^+ : x > -x$
- (b) $\neg \forall x \in \mathbb{Z} : 2x = x + 2$
- (c) $\exists x \in \mathbb{Z} : x^2 = 5$
- (d) $\forall x \in \mathbb{Z} : \exists y \in \mathbb{Z} : y > x + 100$
- (e) $\exists x \in \mathbb{Z} : \forall y \in \mathbb{Z} : (x + y)^2 < x^2$
- (f) $\neg \forall x \in \mathbb{Z} : \exists y \in \mathbb{Z} : x \cdot y = x$

Svar. Sande: (a), (b), (d).

Fremgangsmåde. (a) For positive x er $-x < 0 < x$. Sand.

(b) $2x = x + 2$ kun ved $x = 2$, så $\forall$ -udsagnet er falsk og fornægtelsen sand.

(c) $x^2 = 5$ har ingen heltalsløsning. Falsk.

(d) Hold x fast, vælg $y = x + 101 > x + 100$. Vidnet bygges af x . Sand.

(e) $y = 0$ giver $(x + 0)^2 = x^2$, ikke $< x^2$. Intet x virker for alle y . Falsk.

(f) $y = 1$ giver $x \cdot 1 = x$ for hvert x , så det indre udsagn holder og fornægtelsen er falsk.

MCQ juni 2023, Spm. 34 (flere rigtige)

Domæne $\mathbb{Z}$. Hvilke udsagn er sande? (Et eller flere korrekte svar.)

- (a) $\forall x \in \mathbb{Z} : x^2 > 2x$
- (b) $\neg \exists x \in \mathbb{Z} : x^2 > 2x$
- (c) $\forall x \in \mathbb{Z} : (x < 5 \vee x^2 > 2x)$
- (d) $\forall x \in \mathbb{Z} : \exists y \in \mathbb{Z} : x + y = 2x$
- (e) $\neg \exists x \in \mathbb{Z} : \forall y \in \mathbb{Z} : x + y = 2x$
- (f) $\forall x \in \mathbb{Z} : \forall y \in \mathbb{Z} : x + y = 2x$
- (g) $\neg \exists x \exists y : x + y = 2x \iff \forall x \exists y : x + y \neq 2x$

Svar. Sande: (c), (d), (e).

Fremgangsmåde. (a) Fejler ved $x = 0$ ($0 > 0$ falsk). Falsk.

(b) $x = 3$ giver $9 > 6$, så $\exists$ holder og fornægtelsen er falsk.

(c) Er $x < 5$, holder første led. Er $x \geq 5$, er $x(x - 2) > 0$, så $x^2 > 2x$. Sand.

(d) $y = x$ giver $x + x = 2x$. Sand.

(e) Det indre $\forall y : x + y = 2x$ tvinger $y = x$ for hvert y — umuligt. $\exists x$ er falsk, fornægtelsen sand.

(f) Fejler ved $x = 0, y = 1$. Falsk.

(g) Venstresiden er falsk (vidne $x = y = 0$). Korrekt De Morgan af $\neg \exists x \exists y$ er $\forall x \forall y$, ikke $\forall x \exists y$. Højresiden er sand (vælg $y = x + 1$). $F \iff T$ er falsk.

MCQ juni 2021, Spm. 35 (flere rigtige)

Domæne $\mathbb{Z}$. Hvilke udsagn er sande? (Et eller flere korrekte svar.)

- (a) $\exists a \in \mathbb{Z} : a^2 + 1 = \underline{82}$
- (b) $\forall a \in \mathbb{Z} : a < 2a$
- (c) $\exists a \in \mathbb{Z} : \exists b \in \mathbb{Z} : a = b$
- (d) $\exists a \in \mathbb{Z} : \forall b \in \mathbb{Z} : 2a > b$
- (e) $\forall a \in \mathbb{Z} : \exists b \in \mathbb{Z} : a = 2b$
- (f) $\forall a \in \mathbb{Z} : \exists b \in \mathbb{Z} : a = b + \underline{2}$
- (g) $\forall a \in \mathbb{Z} : \forall b \in \mathbb{Z} : (a < b \vee a > b)$
- (h) $\neg \exists a \forall b : a^2 = b \iff \exists a \forall b : a^2 \neq b$
- (i) $\neg \exists a \in \mathbb{Z} : \forall b \in \mathbb{Z} : a^2 = b$

Svar. Sande: (a), (c), (f), (i).

Fremgangsmåde. (a) $a^2 = 81 \Rightarrow a = \pm 9 \in \mathbb{Z}$. Sand.

(b) $a < 2a \iff 0 < a$, fejler for $a \leq 0$. Falsk.

(c) $a = b = 0$. Sand.

(d) $2a > b$ for alle b kræver b begrænset opadtil, men b løber over hele $\mathbb{Z}$. Falsk.

(e) $a = 2b$ kræver a lige; ulige a (fx 1) fejler. Falsk.

(f) $b = a - 2$ findes altid. Sand.

(g) Fejler når $a = b$. Falsk.

(h) Korrekt fornægtelse af $\exists a \forall b : a^2 = b$ er $\forall a \exists b : a^2 \neq b$, ikke $\exists a \forall b : a^2 \neq b$. Bikonditionalen parrer to udsagn med forskellig sandhedsværdi. Falsk.

(i) $\exists a \forall b : a^2 = b$ kræver ét a , hvis kvadrat er hvert heltal — umuligt. Udsagnet er falsk, fornægtelsen sand.

Eksamen feb 2015, Opg. 4 (DM03 Opg. 4)

$\mathbb{N} = \{0, 1, 2, \dots\}$. Først (4a): hvilke af i og ii er sande?

1. $\forall x \in \mathbb{N} : \exists y \in \mathbb{N} : x < y$
2. $\exists y \in \mathbb{N} : \forall x \in \mathbb{N} : x < y$

Dernæst (4b): angiv fornægtelsen af i uden $\neg$ i svaret.

Svar. 4a: i er sand, ii er falsk. 4b: $\exists x \in \mathbb{N} : \forall y \in \mathbb{N} : x \geq y$.

Fremgangsmåde. 4a-i. Hold x fast, vælg $y = x + 1$. Sand.

4a-ii. Påstår ét y større end hvert x — et største naturligt tal. Tag $x = y$, så fejler $x < y$. Falsk. Samme prædikat, byttet rækkefølge, vendt sandhedsværdi.

4b. Skub $\neg$ ind og vend hver kvantor:

$$\neg(\forall x \exists y : x < y) \equiv \exists x \forall y : \neg(x < y) \equiv \exists x \forall y : x \geq y$$

I ord: “der findes et naturligt tal x større end eller lig med hvert y ” (et største tal). Falsk, som det skal være.

Bevismetoder

Et bevis viser, at en påstand gælder i hvert tilfælde, den dækker. Til eksamen skal du bevise eller modbevise korte påstande om hele og rationale tal, eller vælge det gyldige bevis fra en menu. Tre metoder dækker næsten alt: direkte, kontraposition og modstrid.

Først oversætter du ordene til algebra. Et lige tal er

$$2k$$

et ulige tal er

$$2k + 1$$

for et heltal k . Et rationalt tal er en brøk

$$\frac{a}{b}, \quad a, b \in \mathbb{Z}, \quad b \neq 0.$$

For “irrationel” findes ingen formel. Du antager det modsatte — at tallet **er** rationelt — og rammer en modstrid.

Sådan løser du den

Direkte bevis — for $P \Rightarrow Q$

1. Antag P . Skriv hver hypotese på algebraform: **ulige** $= 2k + 1$, **lige** $= 2k$.
2. Byg udtrykket, påstanden handler om — typisk **summen** eller **produktet**.
3. Reducer. Sæt 2 uden for parentes for **lige**, eller skriv på formen $2 \cdot (\text{heltal}) + 1$ for **ulige**.
4. Konkluder, at udtrykket har den påståede form. Så gælder Q .

Kontraposition — for $P \Rightarrow Q$

1. Skriv den kontraponerede $\neg Q \rightarrow \neg P$. Negér begge sider: f.eks. $\neg Q = \text{“}n \text{ er ulige”}$ og $\neg P = \text{“}n^3 + 5 \text{ er lige”}$.
2. Antag $\neg Q$, og bevis $\neg P$ ved direkte udregning.
3. Det er nok: $\neg Q \rightarrow \neg P$ er logisk det samme som $P \rightarrow Q$.

Modstrid — for “X er irrationel” eller en vilkårlig $P \Rightarrow Q$

1. Antag det modsatte. For “irrationel”: antag at rx **er** rationel, altså $= \frac{c}{d}$ med heltal og $d \neq 0$. For $P \rightarrow Q$: antag $P \wedge \neg Q$.
2. Brug de øvrige rationale hypoteser (på brøkform) til at isolere den faktor, der skulle være irrationel.
3. Vis, at faktoren kommer ud som rationel. Det modsiger antagelsen, som dermed falder.

Et fjerde greb dækker “bevis eller modbevis” om en for-alle-påstand: ét modeksempel slår den ihjel.

Modeksempel først

Ved “bevis eller modbevis” prøv et modeksempel først. Vælg “pæne” værdier, der falder sammen. F.eks. modbeviser $\sqrt{2} \cdot \sqrt{2} = 2$ påstanden om, at produktet af to irrationale tal altid er irrationelt.

Ikke den omvendte

Kontraposition er $\neg Q \rightarrow \neg P$, ikke den omvendte $Q \rightarrow P$. Den omvendte er den hyppigste forkerte MCQ-mulighed. Negér en disjunktion med De Morgan: $\neg(a > \frac{c}{2} \vee b > \frac{c}{2})$ bliver $a \leq \frac{c}{2} \wedge b \leq \frac{c}{2}$, altså et **OG**.

Forskellig fra nul

En “forskellig fra nul”-hypotese bærer ofte hele beviset. I “(rationalt $\neq 0$) $\times$ (irrationelt) er irrationelt” er det $r \neq 0$, der lader dig dividere med r . Drop den, og beviset bryder sammen, for $0 \cdot x = 0$ er rationel.

Tilbagevendende eksamensspørgsmål

DM04 Opg 1 (Rosen 1.7 #1)

Brug et direkte bevis til at vise, at summen af to ulige heltal er lige.

Svar. Sandt. Summen af to ulige tal er $2(a + b + 1)$, altså lige.

Fremgangsmåde. Lad $m = 2a + 1$ og $n = 2b + 1$ for heltal a, b . Så er

$$m + n = (2a + 1) + (2b + 1) = 2a + 2b + 2 = 2(a + b + 1).$$

$a + b + 1$ er et heltal, så $m + n = 2 \cdot (\text{heltal})$ er lige. ■

DM04 Opg 17 (Rosen 1.7 #17)

Vis, at hvis n er et heltal og $n^3 + 5$ er ulige, så er n lige — ved (a) kontraposition, (b) modstrid.

Svar. Begge virker. Ulige n gør n^3 ulige, så $n^3 + 5 = \text{ulige} + \text{ulige} = \text{lige}$.

Fremgangsmåde. (a) Kontraposition. Bevis “hvis n er ulige, så er $n^3 + 5$ lige”. Lad $n = 2k + 1$. Så er n^3 ulige, sig $n^3 = 2j + 1$, og

$$n^3 + 5 = 2j + 6 = 2(j + 3)$$

er lige. Den kontraponerede holder, altså også det oprindelige. ■

(b) Modstrid. Antag $n^3 + 5$ ulige og n ulige. Så er n^3 ulige, og $n^3 + 5 = \text{ulige} + \text{ulige} = \text{lige}$ — modstrid. Altså er n lige. ■

DM04 Opg 41 (Rosen 1.7 #41)

Bevis eller modbevis: produktet af et rationalt tal forskelligt fra nul og et irrationelt tal er irrationelt.

Svar. Sandt. Bevises ved modstrid.

Fremgangsmåde. Lad r være rationel og $\neq 0$ og x irrationel. Skriv $r = \frac{a}{b}$ med heltal $a \neq 0$, $b \neq 0$. Antag for modstrid, at rx er rationel, $rx = \frac{c}{d}$ med heltal c, d og $d \neq 0$. Så er

$$x = \frac{rx}{r} = \frac{c/d}{a/b} = \frac{cb}{da}.$$

Da $a \neq 0$ og $d \neq 0$, er $da \neq 0$, og cb, da er heltal. Så er x rationel. Modstrid. Altså er rx irrationel. ■

MCQ juni 2021, Spm. 36

Angiv et korrekt bevis for: $a + b > c \rightarrow a > \frac{c}{2} \vee b > \frac{c}{2}$.

- (a) $a + b > c \rightarrow a > c - \frac{c}{2} \rightarrow a > \frac{c}{2} \rightarrow a > \frac{c}{2} \vee b > \frac{c}{2}$
- (b) $a + b \leq c \rightarrow a \leq c - b \rightarrow a \leq \frac{c}{2} \vee b \leq \frac{c}{2}$
- (c) $a > \frac{c}{2} \vee b > \frac{c}{2} \rightarrow a + b > \frac{c}{2} + \frac{c}{2} \rightarrow a + b > c$
- (d) $a \leq \frac{c}{2} \wedge b \leq \frac{c}{2} \rightarrow a + b \leq \frac{c}{2} + \frac{c}{2} \rightarrow a + b \leq c$
- (e) $a < \frac{c}{2} \vee b < \frac{c}{2} \rightarrow 2a < c \vee 2b < c \rightarrow a + b < c$

Svar. Mulighed (d): $a \leq \frac{c}{2} \wedge b \leq \frac{c}{2} \rightarrow a + b \leq \frac{c}{2} + \frac{c}{2} \rightarrow a + b \leq c$. Et gyldigt kontrapositionsbevis.

Fremgangsmåde. Her er $P : a + b > c$ og $Q : a > \frac{c}{2} \vee b > \frac{c}{2}$. Ved De Morgan er $\neg Q$ lig $a \leq \frac{c}{2} \wedge b \leq \frac{c}{2}$, og $\neg P$ er $a + b \leq c$. Læg de to uligheder sammen:

$$a + b \leq \frac{c}{2} + \frac{c}{2} = c,$$

altså $\neg P$. Forkast resten: (a) dropper b , (b) kludrer i De Morgan-negationen, (c) beviser den omvendte $Q \rightarrow P$, og (e) bruger skarp $<$ og en forkert negation.

Matematisk induktion

Induktion beviser, at $P(n)$ gælder for alle heltal fra et startpunkt og opefter. Du viser det for det mindste n , og at det gælder for det næste tal når det gælder for ét. Så vælter de som dominobrikker. To dele skal på plads.

Basis: $P(n_0)$ er sand.

Induktionsskridt: $P(k) \Rightarrow P(k+1)$ for alle $k \geq n_0$.

Til eksamen skriver du sjældent et bevis selv. Du får flere kandidat-“beviser” for samme påstand og skal afgøre, hvilke der holder. Du leder efter fejl: en basis der starter forkert, et skridt der går den forkerte vej, eller et algebraskridt der ikke passer.

Sådan løser du den

Skriv et induktionsbevis

1. Skriv $P(n)$ op og find startværdien n_0 , det mindste n påstanden skal gælde for.
2. **Basis.** Sæt $n = n_0$ ind på begge sider og tjek at det stemmer.
3. **Induktionsantagelse (IA).** Antag $P(k)$ sand for et fast $k \geq n_0$.
4. **Induktionsskridt.** Vis $P(k+1)$ med IA. For en sum trækker du det sidste led ud og erstatter resten med IA:

$$5. \quad \sum_{i=1}^{k+1} f(i) = \left(\sum_{i=1}^k f(i) \right) + f(k+1).$$

For en ulighed begrænser du $(k+1)$ -siden med IA og kendte fakta, fx $3^{k+1} = 3 \cdot 3^k$.

6. **Konkluder.** Basis plus induktionsskridt giver $P(n)$ for alle $n \geq n_0$.

Afgør om en kandidat er et gyldigt bevis

1. **Starter basis rigtigt?** Det skal tjekkes ved mindste n , n_0 . Starter det for højt, er der et hul, og beviset er ugyldigt.
2. **Går skridtet rigtig vej?** Det skal udlede $P(k+1)$ ud fra $P(k)$. At udlede det mindre tilfælde fra det større beviser intet.
3. **Bruges IA ved rigtigt indeks?** Antagelsen er kun antaget ved k . At bruge den ved $k+1$ er ulovligt.
4. **Er hvert skridt sandt?** Én forkert lighed eller ulighed dræber beviset. Hold øje med brøctricks som $\frac{1}{2} > \frac{1}{3}$.

Baglæns bevis

Et “bevis” der starter ved målet $P(k+1)$ og arbejder sig ned til hypotesen $P(k)$ antager det, det skulle bevise. Baglæns er ugyldigt, også når hver linje er algebraisk sand.

Gangetrick på IA

At gange IA med en konstant og stiltiende bytte den ud med det næste led er en klassisk fælde. Siderne er ikke ens:

$$4(2^n + 3^n) \neq 2^{n+1} + 3^{n+1}.$$

Tjek at to sider er lig hinanden, før du kæder dem sammen.

Basis ved nul

SDU regner $\mathbb{N} = \{0, 1, 2, \dots\}$, så 0 er med. For “for alle $n \in \mathbb{N}$ ” er basis $n = 0$, ikke $n = 1$. En ekstra basis (fx også $n = 1$) skader ikke.

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 34 (samme som DM547 jan 2019, Spm. 3)

Bevis at $3^n - 1$ er lige for alle $n \in \mathbb{N}$. Hvilke argumenter er gyldige induktionsbeviser? (en eller flere korrekte)

- (a) Basis $3^0 - 1 = 0$ lige. Skridt ($n \geq 0$): $3^{n+1} - 1 = 3 \cdot 3^n - 1 = 3(3^n - 1) + 2 = 3 \cdot 2k + 2 = 2(3k + 1)$, ved IA.
- (b) Basis $3^0 - 1 = 0$ og $3^1 - 1 = 2$ lige. Skridt ($n \geq 2$): $3^n - 1 = 3(3^{n-1} - 1) + 2 = 3 \cdot 2k + 2 = 2(3k + 1)$.
- (c) Basis $3^2 - 1 = 8$ lige. Skridt ($n \geq 3$): $3^n - 1 = 3(3^{n-1} - 1) + 2 = 2(3k + 1)$.
- (d) Basis $3^0 - 1 = 0$ lige. Skridt ($n \geq 0$): $3^n - 1 = 2k \iff 3^n = 2k + 1 \iff 3^{n+1} = 6k + 3 \iff 3^{n+1} - 1 = 2(3k + 1)$.
- (e) Basis $3^0 - 1 = 0$ lige. Skridt ($n \geq 1$): $3^n - 1 = \frac{1}{3} \cdot 3^{n+1} - 1 = \frac{1}{3} \cdot 2k = 2 \cdot \frac{k}{3}$, med $\frac{k}{3} \in \mathbb{Z}$.

Svar. Gyldige: (a), (b) og (d).

Fremgangsmåde. IA: $3^m - 1 = 2k$ for et $k \in \mathbb{Z}$.

- (a) **gyldig.** Basis $n = 0$ korrekt, skridtet går forlæns, 2 sat uden for parentes.
- (b) **gyldig.** Den ekstra basis $n = 1$ er overflødig men harmløs. Skridtet dækker stadig alle n .
- (c) **ugyldig.** Basis starter ved $n = 2$, så $n = 0$ og $n = 1$ dækkes aldrig. Hul.
- (d) **gyldig.** Reversibel kæde fra IA der lander på $3^{n+1} - 1 = 2(3k + 1)$, lige.
- (e) **ugyldig.** Kræver $3^{n+1} - 1 = 2k$, det der skal bevises, ikke IA. Og $\frac{k}{3} \in \mathbb{Z}$ er ubegrundet.

DM547 jan 2021, Spm. 5 (12%)

Påstand: $2^n + 3^n < 4^n$ for $n \geq 2$. Hvilke af 5.a–5.g er gyldige induktionsbeviser?

- (a) Basis $n = 2$. Skridt ganger IA med 4: $4(2^n + 3^n) < 4 \cdot 4^n$, og springer så til $2^{n+1} + 3^{n+1} < 4^{n+1}$.
- (b) Basis $n = 3$, ellers korrekt skridt.
- (c) Basis $n = 2$, skridt $n - 1 \rightarrow n$ med samme gangetrick som 5.a: $4(2^{n-1} + 3^{n-1}) \neq 2^n + 3^n$.
- (d) Skridtet starter ved målet $2^n + 3^n < 4^n$ og udleder hypotesen $2^{n-1} + 3^{n-1} < 4^{n-1}$ (baglæns).
- (e) Skridtet starter ved $2^{n+1} + 3^{n+1} < 4^{n+1}$ (målet) og udleder $2^n + 3^n < 4^n$.
- (f) Basis $n = 2$. Skridt forlæns: $2^{n+1} + 3^{n+1} = 2 \cdot 2^n + 3 \cdot 3^n < 3(2^n + 3^n) < 3 \cdot 4^n < 4 \cdot 4^n = 4^{n+1}$.
- (g) Skrider $2^n + 3^n = \frac{1}{2} \cdot 2^{n+1} + \frac{1}{3} \cdot 3^{n+1}$, og påstår $< \frac{1}{3}(2^{n+1} + 3^{n+1})$.

Svar. Kun (f) er gyldig.

Fremgangsmåde. Basis: $2^n + 3^n < 4^n$ holder først ved $n = 2$ ($13 < 16$), fejler ved $n = 0, 1$. Så $n_0 = 2$.

- (a) **ugyldig.** $4(2^n + 3^n) \neq 2^{n+1} + 3^{n+1}$ (venstresiden er $2^{n+2} + \dots$), så sidste implikation er ubegrundet.
- (b) **ugyldig.** Basis $n = 3$ springer $n = 2$ over.
- (c) **ugyldig.** Samme gangefejl som (a), skrevet med $n - 1 \rightarrow n$.
- (d) **ugyldig.** Går baglæns fra målet til hypotesen.
- (e) **ugyldig.** Antager konklusionen ved $n + 1$ og udleder det mindre tilfælde.
- (f) **gyldig.** Hver ulighed holder: $2 \cdot 2^n < 3 \cdot 2^n$, så IA ved indeks n , så $3 \cdot 4^n < 4 \cdot 4^n$. Forlæns.
- (g) **ugyldig.** Kræver $\frac{1}{2} < \frac{1}{3}$ på 2^{n+1} -leddet (falsk), og bruger IA ved $n + 1$.

MCQ juni 2023, Spm. 35

Hvilke er korrekte induktionsbeviser for $\sum_{i=1}^n \left(\left(\frac{1}{2} \right)^{i-1} - \left(\frac{1}{2} \right)^i \right) = 1 - \left(\frac{1}{2} \right)^n$ for alle $n \geq 1$? (en eller flere korrekte)

- (a) Basis $n = 1, 2$ tjekket; ind.ant.: summen til $k - 1$ er $1 - \left(\frac{1}{2} \right)^{k-1}$ for $k \geq 3$; skridtet ($k \geq 3$) splitter summen til k og indsætter, forlæns $k - 1 \Rightarrow k$.
- (b) Basis $n = 1$; ind.ant.: summen til $k - 1$ er $1 - \left(\frac{1}{2} \right)^{k-1}$ for $k \geq 2$; skridtet udleder summen til $k = 1 - \left(\frac{1}{2} \right)^k$, forlæns $k - 1 \Rightarrow k$.
- (c) Basis $n = 1$; ind.ant.: summen til k er $1 - \left(\frac{1}{2} \right)^k$ for $k \geq 1$; skridtet udleder udsagnet ved $k - 1$ (nedad, $k \Rightarrow k - 1$).

Svar. Gyldige: (a) og (b).

Fremgangsmåde. Identiteten teleskoperer. Indsættelsen i skridtet er:

$$\left(1 - \left(\frac{1}{2} \right)^{k-1} \right) + \left(\left(\frac{1}{2} \right)^{k-1} - \left(\frac{1}{2} \right)^k \right) = 1 - \left(\frac{1}{2} \right)^k.$$

- (a) **gyldig.** Basis $n = 1, 2$ ($n = 2$ overflødig). Forlæns skridt $k - 1 \Rightarrow k$, korrekt algebra.
- (b) **gyldig.** Basis $n = 1$, samme forlæns skridt.
- (c) **ugyldig.** Algebraen er fin, men antager resultatet ved k og går tilbage til $k - 1$. Induktion skal gå opad fra basis.

Relationer, ækvivalens og ordning

En relation på A er en samling ordnede par fra A . Du skriver $(a, b) \in R$ når a er relateret til b :

$$R \subseteq A \times A$$

Til eksamen får du en relation som liste, matrix eller graf. Du skal afgøre hvilke egenskaber den har, om den er ækvivalensrelation eller ordning, eller udregne en lukning eller ækvivalensklasserne. Alt bygger på fire egenskaber og tre definitioner.

Sådan løser du den

Læs relationen som en graf: hvert element er en knude, og $(a, b) \in R$ er en pil fra a til b . De fire egenskaber bliver da til billeder, du kan aflæse.

Tjek de fire egenskaber

1. **Refleksiv.** Hvert element peger på sig selv.

$$(a, a) \in R \text{ for alle } a \in A$$

Graf: løkke på hver knude. Matrix: hele diagonalen er 1. Mangler bare én løkke, er den ikke refleksiv.

2. **Symmetrisk.** Går en pil den ene vej, går den også den anden.

$$(a, b) \in R \implies (b, a) \in R$$

Hver pil skal have sin modsatte. Én pil uden makker vælter det.

3. **Antisymmetrisk.** To forskellige elementer peger aldrig begge veje.

$$(a, b) \in R \wedge (b, a) \in R \implies a = b$$

Find ét par (a, b) og (b, a) med $a \neq b$, så fejler den. Løkker er tilladt.

4. **Transitiv.** Kan du gå $a \rightarrow b \rightarrow c$, skal genvejen $a \rightarrow c$ også være der.

$$(a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R$$

Tjek hver to-trins-kæde for sin genvej. Ét hul vælter det.

Antisymmetrisk er ikke det modsatte af symmetrisk. En relation kan være begge dele (=) eller ingen af delene.

Klassificér relationen

1. **Ækvivalensrelation:** refleksiv, symmetrisk og transitiv. Den deler A op i klasser. Klassen for a er alt, a er relateret til:

$$[a] = \{x \in A \mid (a, x) \in R\}$$

Klasserne overlapper ikke og dækker hele A — en partition.

2. **Partiel ordning:** refleksiv, antisymmetrisk og transitiv. Som ækvivalens, men antisymmetrisk i stedet for symmetrisk. Eksempler: $\leq$ på tal og “går op i” på heltal.

3. **Total ordning:** en partiel ordning hvor alle par kan sammenlignes, altså $(a, b) \in R$ eller $(b, a) \in R$ for ethvert par. $\leq$ er total; “går op i” er ikke (2 og 3 går ikke op i hinanden).

En lukning er den mindste relation, der indeholder R og har en ønsket egenskab. Du tilføjer kun par, fjerner aldrig.

Udregn en lukning

1. **Refleksiv lukning:** tilføj en løkke på hvert element i A .

$$r(R) = R \cup \{(a, a) \mid a \in A\}$$

Husk også løkker på elementer, der ikke optræder i noget par.

2. **Symmetrisk lukning:** tilføj den modsatte af hvert par.

$$s(R) = R \cup R^{-1}$$

Tilføj ikke diagonalløkker — det hører til den refleksive lukning.

3. **Transitiv lukning:** tilføj genveje, til der ikke er flere.

$$t(R) : (a, b), (b, c) \in R \implies \text{tilføj } (a, c)$$

Gentag på den voksende mængde, til intet nyt dukker op. (a, c) er med, hvis der findes en sti fra a til c i grafen.

Relationstyper

= er refleksiv, symmetrisk og antisymmetrisk på én gang — altså både ækvivalensrelation og partiel ordning. “Samme paritet” og “kongruens mod m ” er ækvivalensrelationer; $\leq$ og “går op i” er partielle ordninger.

Antisymmetrisk lukning

Der findes ingen antisymmetrisk lukning. En lukning tilføjer kun par, men antisymmetri kræver, at du fjerner enten (a, b) eller (b, a) når $a \neq b$.

Refleksiv lukning

Refleksiv lukning rammer hvert element i A , også dem uden par. På $A = \{1, 2, 3, 4\}$ med $R = \{(1, 3), (2, 2)\}$ tilføjer du $(1, 1)$, $(3, 3)$ og $(4, 4)$, selvom 4 ikke står i noget par.

Et **Hasse-diagram** tegner en partiel ordning uden støj. Fjern alle løkker og hver kant, der følger af transitivitet; behold kun kanter, hvor intet element ligger strengt imellem. Tegn det mindste element nederst og udelad pilespidser. Et element er maksimalt, hvis ingen linje går opad fra det, og minimalt, hvis ingen går nedad.

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 35 (flere korrekte)

Lad $R = \{(a, a), (a, b), (b, a), (c, c)\}$ være en relation på $\{a, b, c\}$. Hvilke udsagn er sande?

- (a) R er reflektiv
- (b) R er symmetrisk
- (c) R er antisymmetrisk
- (d) R er transitiv
- (e) R er en ækvivalensrelation
- (f) R er en partiel ordning

Svar. Kun (b), R er symmetrisk.

Fremgangsmåde. Reflektiv: (b, b) mangler, nej. Symmetrisk: (a, b) har (b, a) , og løkkerne er deres egne makker, ja. Antisymmetrisk: (a, b) og (b, a) med $a \neq b$, nej. Transitiv: (b, a) og (a, b) kræver (b, b) , som mangler, nej. Ækvivalens og partiel ordning fejler dermed begge.

MCQ juni 2025, Spm. 36 (flere korrekte)

Lad $A = \{a, b, c, d\}$. Hvilke udsagn er sande?

- (a) Den reflexive lukning af $\emptyset$ på A er $\{(a, a), (b, b), (c, c), (d, d)\}$
- (b) Den reflexive lukning af $\{(a, b), (b, c)\}$ på A er $\{(a, c)\}$
- (c) Den symmetriske lukning af $\{(a, a), (a, b), (c, d)\}$ på A er $\{(a, a), (a, b), (b, a), (b, b), (c, c), (c, d), (d, c), (d, d)\}$
- (d) Den symmetriske lukning af $\{(a, b), (b, a), (b, c)\}$ på A er $\{(a, b), (b, a)\}$
- (e) Den transitive lukning af $\{(a, b), (b, c), (c, d)\}$ på A er $\{(a, b), (a, c), (a, d), (b, c), (b, d), (c, d)\}$
- (f) Den transitive lukning af $\{(a, b), (c, d)\}$ på A er $\{(a, b), (c, d)\}$

Svar. (a), (e) og (f).

Fremgangsmåde. (a): reflexiv lukning af den tomme relation er præcis diagonalen, passer. (b): reflexiv lukning tilføjer løkker, ikke (a, c) , falsk. (c): symmetrisk lukning tilføjer ikke løkkerne $(b, b), (c, c), (d, d)$, falsk. (d): mangler (b, c) og (c, b) , falsk. (e): genvejene $(a, c), (b, d), (a, d)$ langs kæden $a \rightarrow b \rightarrow c \rightarrow d$, passer. (f): ingen par kan kædes sammen, uændret, passer.

MCQ juni 2023, Spm. 36 (3 point)

Betragt relationen på $\{a, b, c\}$: $R = \{(a, b), (b, a), (b, b), (b, c)\}$. Angiv den transitive lukning af R .

Svar. $t(R) = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c)\}$.

Fremgangsmåde. b når a, b og c . Da (a, b) gælder, når a alt det b når: $(a, b) + (b, a)$ giver (a, a) , $(a, b) + (b, c)$ giver (a, c) , $(a, b) + (b, b)$ giver (a, b) . c har ingen udgående kanter, så ingen nye par.

DM547 januar 2021, Spørgsmål 7 (3%)

Angiv den symmetriske lukning af $T = \{(a, b), (b, b), (c, d), (d, e)\}$.

Svar. $s(T) = \{(a, b), (b, a), (b, b), (c, d), (d, c), (d, e), (e, d)\}$.

Fremgangsmåde. $s(T) = T \cup T^{-1}$: tilføj den modsatte af hvert par, altså (b, a) , (d, c) og (e, d) . (b, b) er sin egen modsatte. Tilføj ikke diagonalløkker, kæd ikke par sammen.

DM547 januar 2021, Spørgsmål 6 (8%)

En digraf for relationen S på $\{a, b, c, d, e, f\}$ har løkker på a, b, e, f , kanterne $a \rightarrow b$, $a \rightarrow c$, $a \rightarrow d$, $a \rightarrow e$, $b \rightarrow e$, $c \rightarrow d$, $f \rightarrow d$, og tovejsparret $e \leftrightarrow f$. Hvilke af følgende gælder: reflektiv, symmetrisk, antisymmetrisk, transitiv, ækvivalensrelation, partiel ordning, total ordning?

Svar. Ingen af dem.

Fremgangsmåde. Reflektiv: c og d har ingen løkke, nej. Symmetrisk: $a \rightarrow b$ uden $b \rightarrow a$, nej. Antisymmetrisk: $e \leftrightarrow f$ med $e \neq f$, nej. Transitiv: $e \rightarrow f$ og $f \rightarrow d$, men intet $e \rightarrow d$, nej. Når alle fire grundegenskaber fejler, fejler de tre ordningstyper også.

SE4-DMAD juni 2024, Spm. 6 (4%)

På $\{a, b, c\}$, hvilke af disse er ækvivalensrelationer?

- (a) $\{(a, a), (b, b), (c, c)\}$
- (b) $\{(a, a), (a, b), (b, a), (b, b)\}$
- (c) $\{(a, a), (a, b), (b, b), (c, c)\}$
- (d) $\{(a, a), (a, b), (b, b), (b, c), (c, c)\}$
- (e) $\{(a, a), (a, b), (b, a), (b, b), (c, c)\}$

Svar. (a) og (e).

Fremgangsmåde. Skal være reflektiv, symmetrisk og transitiv på $\{a, b, c\}$. (a): bare diagonalen, alle tre holder. (b): (c, c) mangler, ikke reflektiv. (c): (a, b) uden (b, a) , ikke symmetrisk. (d): (a, b) uden (b, a) , ikke symmetrisk. (e): reflektiv, symmetrisk via $(a, b), (b, a)$, og transitiv ($\{a, b\}$ som blok plus c alene).

Klasserne i (e): $[a] = [b] = \{a, b\}$ og $[c] = \{c\}$, disjunkte og dækkende.

Del II — Algoritmer og datastrukturer

Asymptotisk analyse

Asymptotisk analyse sammenligner funktioner efter hvor hurtigt de vokser for stort n . Konstante faktorer og små n er ligegyldige; kun forholdet mellem to funktioner, når $n \rightarrow \infty$, tæller.

Eksamen spørger på to måder: om en påstand som “ $f(n)$ er $O(g(n))$ ” er sand, eller om Θ -køretiden for en løkke. Begge løses med samme greb: kig på forholdet $f(n)/g(n)$.

De fem symboler er bare fem måder at sige “hvor hurtigt vokser f i forhold til g ”. Med køretid i baghovedet:

Symbol	Hverdagsord	Hvad det siger om køretiden
$O(g)$	“højst” — loft	Bliver aldrig værre end g (på nær konstanter). Værstefald-grænse.
$\Omega(g)$	“mindst” — gulv	Er mindst lige så slem som g . En nedre grænse.
$\Theta(g)$	“præcis”	Både loft og gulv er g — vokser nøjagtig lige så hurtigt som g .
$o(g)$	“skarpt under”	Strengt langsommere end g ; et loft den aldrig når.
$\omega(g)$	“skarpt over”	Strengt hurtigere end g ; et gulv den aldrig når.

Streng vs. ikke-streng

O og Ω tillader lighed (loft/gulv må røres), o og ω gør ikke (strengt under/over). Θ er O og Ω på én gang. Derfor: hver gang Θ holder, holder O og Ω også — men ikke omvendt.

Sådan løser du den

En påstand som “ $f(n)$ er $O(g(n))$ ” spørger kun om én ting: **vokser f højst lige så hurtigt som g** ? Det eneste værktøj du behøver er vækststigen — langsomt til venstre, hurtigt til højre:

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n^n.$$

To huskeregler dækker næsten alt: en **eksponentiel** ($2^n, 3^n$) slår altid en **potens** ($n^2, n^3, \dots$), og en **log-faktor** rykker dig kun en lillebitte smule på stigen.

Konstanter

Det yderste 1 på stigen er **alle konstanter**: 2, 3, 100 vokser ikke med n , så de sidder alle på samme plads. Derfor er enhver konstant O (endda Θ) af enhver anden — fx er $3 = O(2)$ sand, og $O(2)$ betyder bare $O(1)$.

Afgør en påstand f er O / Ω / Θ af g

- Find f (venstre) og g (inde i parentes).
- Placér begge på vækststigen — hvem står længst til højre (vokser hurtigst)?
- Slå relationen op: står f til **venstre for** g , er $f = O(g)$ og $o(g)$; **samme plads**, $\Theta(g)$; **højre for**, $\Omega(g)$ og $\omega(g)$.

For et hurtigt blik: er f under eller lig med g på stigen, holder “ $f = O(g)$ ”; er f over g , gør den ikke.

Den præcise metode (hvis stigen ikke rækker): del de to funktioner og se hvad forholdet går mod langt ude. Lad

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}.$$

Grænseværdien L giver relationen (en sum beholder kun sit hurtigste led, så forenkl først).

Fra grænseværdi til relation:

$$\begin{aligned} L \rightarrow c > 0 &\Rightarrow f = \Theta(g) \\ L \rightarrow 0 &\Rightarrow f = o(g) \text{ (og dermed } O(g) \text{)} \\ L \rightarrow \infty &\Rightarrow f = \omega(g) \text{ (og dermed } \Omega(g) \text{)} \end{aligned}$$

Tænk på symbolerne som tal-sammenligninger: O er $\leq$, Ω er $\geq$, Θ er $=$, o er $<$, ω er $>$. Θ kræver et positivt endeligt L . O holder så længe forholdet ikke vokser uden grænse, så det dækker også $L \rightarrow 0$.

Hele opslaget på én tabel — regn $L = \lim f(n)/g(n)$ og slå op:

Påstand	Betyder	Sand når L er	Eksempel (sand)
$f = O(g)$	$f \leq g$	0 eller en konstant	$n = O(n^2)$
$f = o(g)$	$f < g$	0	$n = o(n^2)$
$f = \Theta(g)$	$f = g$	en konstant > 0	$3n = \Theta(n)$
$f = \Omega(g)$	$f \geq g$	en konstant eller ∞	$n^2 = \Omega(n)$
$f = \omega(g)$	$f > g$	∞	$n^2 = \omega(n)$

Kort sagt: er f **under** g på vækststigen, så er $f = O(g)$ (og $o(g)$). **Over:** $\Omega(g)$ (og $\omega(g)$). **Samme plads:** $\Theta(g)$ — og så gælder både O og Ω samtidig.

Polynomium vs. eksponentiel

To kendsgerninger afgør næsten alt. Ethvert polynomium slår enhver eksponentialfunktion: $n^a/b^n \rightarrow 0$ for $a > 0$ og $b > 1$. Og enhver rod slår enhver polylog: $(\log n)^a/n^d \rightarrow 0$ for $a, d > 0$.

Konstante faktorer

Konstante faktorer og summer af samme grad ændrer ikke klassen: $n + n + n = \Theta(n/3) = \Theta(n)$. Men en log-faktor tæller. $(\log n)^3$ er ikke $\Theta(3 \log n)$, fordi $(\log n)^3/(3 \log n) = (\log n)^2/3 \rightarrow \infty$.

For løkker tæller du to ting hver for sig: hvor mange gange den ydre løkke kører, og arbejdet indeni per gennemløb.

Find Θ -køretiden for en løkke

1. Tæl ydre gennemløb. Additivt skridt ($i = i + \underline{c}$) giver $\Theta(n)$; multiplikativt ($i = \underline{2} \cdot i$) giver $\Theta(\log n)$.

- Find det indre arbejde per ydre gennemløb. Tjek om en tæller nulstilles hvert gennemløb eller bliver ved med at stige.
- Gang ydre antal med indre omkostning, smid konstanter væk, skriv Θ .

Indre tæller

Sættes en indre tæller én gang uden for begge løkker og kun stiger, er det samlede indre arbejde $\Theta(n)$ for hele kørslen, ikke per gennemløb. Det laver et tilsyneladende $\Theta(n^2)$ om til $\Theta(n)$.

Tilbagevendende eksamensspørgsmål

MCQ juni 2023, Spm. 5

Hvilke af følgende er sande? (Et eller flere svar.)

- (a) $\underline{n}$ er $O(\sqrt{n})$
- (b) $\underline{n + n}$ er $O(n \log n)$
- (c) $\underline{n \log n}$ er $O(n^{3/2})$
- (d) $\underline{(\log n)^2}$ er $O(n^{1/2})$
- (e) $\underline{3^n}$ er $O((\log n)^3)$
- (f) $\underline{2^n \log n}$ er $O(2^n n)$
- (g) $\underline{n^{1/7}}$ er $O(\log(n^{17}))$

Svar. (b), (c), (d) og (f) er sande.

Fremgangsmåde. Tag hvert forhold mod uendelig. (a) $n/\sqrt{n} = \sqrt{n} \rightarrow \infty$, falsk. (b) $2n/(n \log n) = 2/\log n \rightarrow 0$, sand. (c) $n \log n/n^{1.5} = \log n/\sqrt{n} \rightarrow 0$, sand. (d) $(\log n)^2/\sqrt{n} \rightarrow 0$ (rod slår polylog), sand. (e) 3^n mod en polylog $\rightarrow \infty$, falsk. (f) $\log n/n \rightarrow 0$, sand. (g) $n^{1/7}$ mod $17 \log n \rightarrow \infty$, falsk. Et forhold mod 0 tæller stadig som O .

MCQ juni 2023, Spm. 6

Hvilke af følgende er sande? (Et eller flere svar.)

- (a) $\underline{n}$ er $\Omega((\log n)^2)$
- (b) $\underline{4^n}$ er $\omega(2^n)$
- (c) $\underline{n + n + n}$ er $\Theta(n/3)$
- (d) $\underline{(\log n)^3}$ er $\Theta(3 \log n)$
- (e) $\underline{n^2/\log n}$ er $o(n^2 \log n)$
- (f) $\underline{n^{1.5} + n^{2.0}}$ er $\Theta(n^{1.75})$
- (g) $\underline{2^n}$ er $o(n^n)$

Svar. (a), (b), (c), (e) og (g) er sande.

Fremgangsmåde. (a) $n/(\log n)^2 \rightarrow \infty$, så Ω holder. (b) $4^n/2^n = 2^n \rightarrow \infty$, strengt hurtigere, ω holder. (c) $3n/(n/3) = 9$, positiv konstant, Θ holder. (d) $(\log n)^3/(3 \log n) = (\log n)^2/3 \rightarrow \infty$, ikke konstant, Θ fejler. (e) $1/(\log n)^2 \rightarrow 0$, o holder. (f) summen opfører sig som n^2 , og $n^2/n^{1.75} \rightarrow \infty$, Θ fejler. (g) $(2/n)^n \rightarrow 0$, o holder.

MCQ juni 2023, Spm. 25

Hvad er den asymptotiske køretid i Θ -notation?

ALGORITME3(n): $i = 1$; while $i \leq n$: { $j = n$; while $j > 1$: $j = j - 1$; $i = 2*i$ }

- (a) $\Theta(\log n)$
- (b) $\Theta(\sqrt{n})$
- (c) $\Theta(n)$
- (d) $\Theta(n \log n)$
- (e) $\Theta(n^2)$
- (f) $\Theta(n^3)$

Svar. (d) $\Theta(n \log n)$.

Fremgangsmåde. Ydre løkke: $i = 1, 2, 4, \dots$ indtil $i > n$, altså $\Theta(\log n)$ gennemløb. Indre løkke: j tæller fra n ned til 2, det er $n - 1$ skridt hvert gennemløb uafhængigt af i , altså $\Theta(n)$. I alt $\Theta(\log n) \cdot \Theta(n) = \Theta(n \log n)$.

MCQ juni 2023, Spm. 24

Hvad er den asymptotiske køretid i Θ -notation?

ALGORITME2(n): $i = 1$; $j = 1$; while $i \leq n$: { $i = i + 5$; while $j < i$: $j = j + 1$ }

- (a) $\Theta(\log n)$
- (b) $\Theta(\sqrt{n})$
- (c) $\Theta(n)$
- (d) $\Theta(n \log n)$
- (e) $\Theta(n^2)$
- (f) $\Theta(n^3)$

Svar. (c) $\Theta(n)$. Det er fælde-tilfældet.

Fremgangsmåde. j sættes til 1 én gang uden for begge løkker og nulstilles aldrig. Den ydre løkke kører cirka $n/5$ gange. Den indre løkke skubber kun j opad, så hen over hele kørslen klatrer j fra 1 op til cirka n — det er $\Theta(n)$ indre skridt i alt, ikke per gennemløb. I alt $\Theta(n/5) + \Theta(n) = \Theta(n)$. At læse det som $\Theta(n^2)$ er fejlen spørgsmålet jagter.

Rekursionsligninger og Master Theorem

En Divide-and-Conquer-algoritme deler problemet op, løser delene rekursivt og samler resultatet. Køretiden $T(n)$ står derfor udtrykt ved sig selv på mindre input. Gør algoritmen a kald, hvert på et stykke af størrelse $\frac{n}{b}$, og bruger $f(n)$ på at dele op og samle, så er

$$T(n) = aT\left(\frac{n}{b}\right) + f(n).$$

Master Theorem giver en lukket Θ -grænse for $T(n)$.

Sådan løser du den

Sammenlign arbejdet i rekursionen, målt af n^α med

$$\alpha = \log_b a,$$

mod arbejdet uden for, $f(n)$. Den, der vokser hurtigst, bestemmer svaret.

Skelseksponenten

Skelseksponenten $\alpha = \log_b a$ er det samme som p i eksamenens svar: skriver opgaven $\Theta(n^p)$, så er $p = \alpha = \log_b a$. Du regner altså p ved at tage log af a med grundtal b — fx $T(n) = 5T(\frac{n}{2}) + n^2$ giver $p = \log_2 5$. Pas på rækkefølgen: det er $\log_b a$, ikke $\log_a b$.

Hvornår p optræder

p optræder kun når svaret er n^α (tilfælde 1). Vinder $f(n)$ (tilfælde 3), er svaret $f(n)$ selv — fx $\Theta(n^{\frac{1}{2}})$ — uden noget p . Tømmefinger: ser du “ $p = \log_b a$ ”, er det tilfælde 1; ser du en ren funktion som $n^{\frac{1}{2}}$ eller n^2 , er det tilfælde 3.

Master Theorem (Cormen et al., 4. udg.)

1. Aflæs a , b og $f(n)$ fra ligningen. Kun de tre skifter fra år til år.
2. Regn skelseksponenten $\alpha = \log_b a$ ud, og skriv skelfunktionen n^α .
3. Hold $f(n)$ op mod n^α og find tilfældet.
4. Skriv den Θ -grænse, tilfældet giver.

De tre udfald handler kun om: er n^α eller $f(n)$ størst?

Hvem er størst?	Eksempel ($n^\alpha = n$)	Tilfælde	Svar $T(n)$
n^α størst	$f(n) = \sqrt{n}$	1	$\Theta(n^\alpha)$
lige store	$f(n) = n$	2	$\Theta(n^\alpha \log n)$
$f(n)$ størst — <i>en hel potens</i>	$f(n) = n^2$	3	$\Theta(f(n))$
$f(n)$ større, men <i>kun et log</i>	$f(n) = n \log n$	—	kan ikke løses

For at se **hvem der er størst**, placér både n^α og $f(n)$ på vækststigen — længst til højre vinder:

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^2 \log n < n^3 < 2^n$$

Potenser på stigen

Et n^c med større potens slår altid et med mindre ($n^2 > n^{1.5} > n$), uanset log-faktorer: ethvert n^c slår $\log n$, og enhver eksponentiel (2^n) slår alle n^c . Til Master Theorem: er n^α længere til højre end $f(n)$, er det tilfælde 1; står de på samme plads, tilfælde 2; er $f(n)$ en **hel potens** længere til højre, tilfælde 3. Bemærk n^α kan have skæv potens, fx $n^{\log_2 5} \approx n^{2.32}$, som ligger mellem n^2 og n^3 .

Hullet i tilfælde 3

Tilfælde 3 kræver, at $f(n)$ er en **hel potens** større end n^α (et helt trin på stigen, fx $n^2 \bmod n$). Er $f(n)$ kun en log-faktor større — som $f(n) = n \log n$ når $n^\alpha = n$ — så er den for stor til tilfælde 2 og for lille til tilfælde 3. Den falder i hullet, og Master Theorem kan **ikke** løse den.

Tilfælde 1 — n^α er størst. Rekursionen vinder, og svaret er n^α .

$$\text{Tilfælde 1: } f(n) = O(n^{\alpha-\varepsilon}) \implies T(n) = \Theta(n^\alpha).$$

Tilfælde 2 — de er lige store. Uafgjort, så du lægger et $\log n$ oveni.

$$\text{Tilfælde 2: } f(n) = \Theta(n^\alpha) \implies T(n) = \Theta(n^\alpha \log n).$$

Tilfælde 3 — $f(n)$ er størst. Toparbejdet vinder, og svaret er $f(n)$.

$$\text{Tilfælde 3: } f(n) = \Omega(n^{\alpha+\varepsilon}) \implies T(n) = \Theta(f(n)).$$

I tilfælde 3 skal du tjekke regularitetsbetingelsen: at et $c < 1$ opfylder $a f(\frac{n}{b}) \leq c f(n)$ for store n . For polynomielle f holder den altid.

Fast svarmenu

MCQ'en har samme faste svarmenu hvert år: $\Theta(1)$, $\Theta(\log n)$, $\Theta(n^{\log_4 3})$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^{\log_3 4})$, $\Theta(n^2)$, $\Theta(n^2 \log n)$, $\Theta(n^3)$ og “kan ikke løses med Master Theorem”. Løs ligningen, og find svaret i menuen.

Tre situationer giver svaret “**kan ikke løses med Master Theorem**”:

Fælde	Eksempel	Hvorfor
Hullet (log-faktor)	$2T(\frac{n}{2}) + n \log n$	$f(n)$ kun et log større end n^α — for stor til tilfælde 2, for lille til tilfælde 3
Ulige stykker	$T(\frac{n}{3}) + T(2\frac{n}{3}) + n$	kaldene har ikke samme størrelse $\frac{n}{b}$
Subtraktion	$2T(n-2) + n$	problemet divideres ikke ($T(n-c)$, ikke $T(\frac{n}{b})$) — der findes intet b

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 1 (samme type 2015–2023)

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = \underline{2} \cdot T(n/\underline{4}) + \underline{n^2}$

- (a) $T(n) = \Theta(1)$
- (b) $T(n) = \Theta(\log n)$

- (c) $T(n) = \Theta(n^{\log_4 3})$
- (d) $T(n) = \Theta(n)$
- (e) $T(n) = \Theta(n \log n)$
- (f) $T(n) = \Theta(n^{\log_3 4})$
- (g) $T(n) = \Theta(n^2)$
- (h) $T(n) = \Theta(n^2 \log n)$
- (i) $T(n) = \Theta(n^3)$
- (j) Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Svar. Mulighed (g): $T(n) = \Theta(n^2)$ — tilfælde 3.

Fremgangsmåde. $a = \underline{2}$, $b = \underline{4}$, $f(n) = \underline{n^2}$. Skelseksponenten er

$$\alpha = \log_4 2 = 0.5, \quad n^\alpha = n^{0.5}.$$

$f(n) = n^2$ er polynomielt større end $n^{0.5}$ (med $\varepsilon = 1.5$): tilfælde 3. Regularitet: $a f(\frac{n}{b}) = 2(\frac{n}{4})^2 = \frac{1}{8}n^2 \leq c n^2$ med $c = \frac{1}{8} < 1$. Den holder, så $T(n) = \Theta(n^2)$.

MCQ juni 2025, Spm. 2

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = \underline{4} \cdot T(n/\underline{2}) + \underline{n^2}$

- (a) $T(n) = \Theta(1)$
- (b) $T(n) = \Theta(\log n)$
- (c) $T(n) = \Theta(n^{\log_4 3})$
- (d) $T(n) = \Theta(n)$
- (e) $T(n) = \Theta(n \log n)$
- (f) $T(n) = \Theta(n^{\log_3 4})$
- (g) $T(n) = \Theta(n^2)$
- (h) $T(n) = \Theta(n^2 \log n)$
- (i) $T(n) = \Theta(n^3)$
- (j) Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Svar. Mulighed (h): $T(n) = \Theta(n^2 \log n)$ — tilfælde 2.

Fremgangsmåde. $a = \underline{4}$, $b = \underline{2}$, $f(n) = \underline{n^2}$. Skelseksponenten er

$$\alpha = \log_2 4 = 2, \quad n^\alpha = n^2.$$

$f(n) = n^2$ er lige så stor som n^α , altså uafgjort med $k = 0$: tilfælde 2, $T(n) = \Theta(n^2 \log n)$.

MCQ juni 2025, Spm. 3

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = \underline{4} \cdot T(n/\underline{3}) + \underline{n}$

- (a) $T(n) = \Theta(1)$
- (b) $T(n) = \Theta(\log n)$
- (c) $T(n) = \Theta(n^{\log_4 3})$
- (d) $T(n) = \Theta(n)$
- (e) $T(n) = \Theta(n \log n)$
- (f) $T(n) = \Theta(n^{\log_3 4})$
- (g) $T(n) = \Theta(n^2)$

- (h) $T(n) = \Theta(n^2 \log n)$
 (i) $T(n) = \Theta(n^3)$
 (j) Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Svar. Mulighed (f): $T(n) = \Theta(n^{\log_3 4})$ — tilfælde 1.

Fremgangsmåde. $a = 4$, $b = 3$, $f(n) = n$. Skelseksponenten er

$$\alpha = \log_3 4 \approx 1.26.$$

Her vokser n^α hurtigere end $f(n) = n$. f er polynomielt mindre: tilfælde 1, $T(n) = \Theta(n^{\log_3 4})$.

MCQ juni 2025, Spm. 4

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = T(n/4) + 1$

- (a) $T(n) = \Theta(1)$
 (b) $T(n) = \Theta(\log n)$
 (c) $T(n) = \Theta(n^{\log_4 3})$
 (d) $T(n) = \Theta(n)$
 (e) $T(n) = \Theta(n \log n)$
 (f) $T(n) = \Theta(n^{\log_3 4})$
 (g) $T(n) = \Theta(n^2)$
 (h) $T(n) = \Theta(n^2 \log n)$
 (i) $T(n) = \Theta(n^3)$
 (j) Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Svar. Mulighed (b): $T(n) = \Theta(\log n)$ — tilfælde 2.

Fremgangsmåde. Her er $a = 1$, $b = 4$, $f(n) = 1$. Skelseksponenten er

$$\alpha = \log_4 1 = 0, \quad n^\alpha = n^0 = 1.$$

$f(n) = 1$ er lige så stor som n^0 : tilfælde 2, $T(n) = \Theta(\log n)$.

MCQ juni 2025, Spm. 2 (samme menu)

Hvilket af nedenstående svar gælder for følgende rekursionsligning? $T(n) = 5 \cdot T(n/5) + n \log n$

- (a) $T(n) = \Theta(n^p)$ med $p = \log_5 5$
 (b) $T(n) = \Theta(n^p \log n)$ med $p = \log_5 5$
 (c) Rekursionsligningen kan ikke løses med Master Theorem.

Svar. Mulighed (c): rekursionsligningen **kan ikke løses** med Master Theorem.

Fremgangsmåde. $a = 5$, $b = 5$, $f(n) = n \log n$. Skelseksponenten er

$$\alpha = \log_5 5 = 1, \quad n^\alpha = n.$$

Sammenlign $f(n) = n \log n$ mod n : den er **større** end n (ikke tilfælde 1 eller 2), men kun en log-faktor større — ikke en hel potens, så $f(n)$ er ikke $\Omega(n^{1+\varepsilon})$ (ikke tilfælde 3). Den falder i hullet mellem tilfælde 2 og 3, og kan derfor ikke løses.

Master-sætningen

Samme skabelon, fire udfald. Regn $\alpha = \log_b a$, og lad $f(n)$ dyste mod n^α : vinder n^α , tilfælde 1; uafgjort, et ekstra $\log n$ (tilfælde 2); vinder f , tilfælde 3.

Sortering og udvælgelse

At sortere er at lægge n tal i rækkefølge. Der findes to slags algoritmer, og forskellen afgør alt til eksamen.

De **sammenligningsbaserede** (insertion, selection, merge, quick, heap, tree-sort) spørger “er a mindre end b ?”. Værdiernes størrelse er ligegyldig; kun antallet tæller, så køretiden afhænger kun af n .

De **distributionsbaserede** (counting, radix) sammenligner aldrig. De bruger værdierne som indeks i et array. Det kommer under sammenligningsgrænsen, men køretiden afhænger nu af værdiernes størrelse.

Enhver sammenligningssortering bruger mindst $\Omega(n \log n)$ sammenligninger i værste fald:

$$\text{højde af decision tree} \geq \log(n!) = \Omega(n \log n)$$

Et decision tree har ét blad per rækkefølge, altså $n!$ blade, og et binært træ med $n!$ blade er mindst så højt. Vil du hurtigere ned, må du holde op med at sammenligne — det er netop, hvad counting og radix gør.

Sådan løser du den

Typisk spørgsmål: “sortér n heltal i et givet interval — hvad er værste-falds-køretiden for algoritme X ?”

Find køretiden for en sorteringsalgoritme

1. Læs inputtet: antal elementer n og værdiområde $[0, n^3]$ (for radix: antal cifre og cifferområde).
2. Sammenligningsbaseret? Så er værdiområdet et vildspor; køretiden afhænger kun af n — slå den op nedenfor.
3. Distributionsbaseret (counting, radix)? Sæt værdiområdet ind i formlen og tag det dominerende led.

Algoritme	Værste fald	In-place	Note
Insertion sort	$\Theta(n^2)$	ja	Bedste fald $\Theta(n)$ på sorteret input.
Selection sort	$\Theta(n^2)$	ja	Altid $\Theta(n^2)$, også på sorteret input.
Merge sort	$\Theta(n \log n)$	nej	Altid $\Theta(n \log n)$. Bruger et ekstra array.
Quicksort	$\Theta(n^2)$	ja	Typisk $\Theta(n \log n)$; værst på sorteret input.
Heapsort	$\Theta(n \log n)$	ja	Build-heap $\Theta(n)$, derefter n heapify.
Counting sort	$\Theta(n + k)$	nej	k er værdiområdet. Stabil.
Radix sort	$\Theta(d(n + k))$	nej	d gennemløb af counting sort, k er cifferområdet.

Counting sort tæller hver værdis forekomster, summerer tællingerne til en prefix-sum og placerer elementerne bagfra, så ens værdier holder rækkefølge (stabilitet):

```
COUNTING-SORT(A, n, k)
  for i = 0 to k: C[i] = 0
```

```

for j = 1 to n: C[A[j]] += 1      // C[i] = antal af værdien i
for i = 1 to k: C[i] += C[i-1]  // C[i] = antal værdier <= i
for j = n downto 1:             // bagfra => stabil
    B[C[A[j]]] = A[j]
    C[A[j]] -= 1
return B

```

Radix sort kører counting sort per ciffer, fra mindst til mest betydende. Hvert af de d gennemløb koster $\Theta(n + k)$ med k lig cifferområdet:

$$T(n) = \Theta(d(n + k))$$

Quicksort vælger en pivot, deler arrayet i mindre og større og sorterer hver del. CLRS bruger Lomuto-partition med sidste element som pivot. Bagefter sidder pivoten på sin endelige plads med mindre-eller-lig til venstre, større til højre:

```

PARTITION(A, p, r)
x = A[r]; i = p - 1
for j = p to r - 1:
    if A[j] <= x: i += 1; swap A[i], A[j]
swap A[i+1], A[r]
return i + 1

```

Værdiområde

Værdiområdet i opgaveteksten er ofte med for at narre dig. For en sammenligningssortering er det støj: quicksort er $\Theta(n^2)$ i værste fald, uanset om tallene ligger i $[0, n)$ eller $[0, n^9)$. Det betyder kun noget for counting og radix.

Counting sort

Counting sort er ikke altid lineær. Med $k = \underline{n^2}$ koster den $\Theta(n + n^2) = \Theta(n^2)$. Den er kun $O(n)$, når $k = O(n)$. Deler du heltallet op i cifre med radix sort, holdes cifferområdet lille, og du slipper med $\Theta(n)$.

Tilbagevendende eksamensspørgsmål

MCQ juni 2023, Spm. 27

Vi betragter sortering af $\underline{n}$ heltal med værdier i intervallet $[0, n^3)$. Hvad er værste-falds-køretiden for CountingSort på dette input?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^3)$

Svar. (d) $\Theta(n^3)$.

Fremgangsmåde. CountingSort koster $\Theta(n + k)$. Her er $k = \underline{n^3}$, og n^3 dominerer:

$$\Theta(n + n^3) = \Theta(n^3)$$

MCQ juni 2023, Spm. 28

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^3]$. Hvad er værste-falds-køretiden for RadixSort, når heltallene behandles som tre cifre med værdier i intervallet $[0, n]$?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^3)$

Svar. (a) $\Theta(n)$.

Fremgangsmåde. $d = 3$ gennemløb, hver en counting sort med cifferområde $k = n$, koster $\Theta(n + n) = \Theta(n)$. Konstanten $d = 3$ forsvinder:

$$3 \cdot \Theta(n) = \Theta(n)$$

MCQ juni 2023, Spm. 29

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^3]$. Hvad er værste-falds-køretiden for QuickSort på dette input?

- (a) $\Theta(n)$
- (b) $\Theta(n \log n)$
- (c) $\Theta(n^2)$
- (d) $\Theta(n^3)$

Svar. (c) $\Theta(n^2)$.

Fremgangsmåde. QuickSort er sammenligningsbaseret, så værdiområdet er irrelevant. Værste fald er maksimalt ubalancerede partitioner ($n - 1$ og 0):

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$$

MCQ juni 2019, Spm. 27

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^2]$. Ved TreeSort menes algoritmen, der indsætter tallene ét ad gangen i et søgetræ og derefter laver et inorder-gennemløb. Hvilke af algoritmerne nedenfor har værste-falds-køretid $\Theta(n^2)$? (Et eller flere svar.)

- (a) CountingSort
- (b) RadixSort, hvor heltallene behandles som to cifre med værdier i intervallet $[0, n]$
- (c) MergeSort
- (d) TreeSort, hvor søgetræet er et rød-sort træ
- (e) TreeSort, hvor søgetræet er et ubalanceret søgetræ
- (f) QuickSort
- (g) InsertionSort

Svar. (a) CountingSort, (e) ubalanceret TreeSort, (f) QuickSort og (g) InsertionSort.

Fremgangsmåde. CountingSort: $k = n^2$ dominerer, $\Theta(n^2) \rightarrow$ ja. RadixSort, to cifre i base n : to gennemløb $\rightarrow \Theta(n) \rightarrow$ nej. MergeSort: altid $\Theta(n \log n) \rightarrow$ nej. Rød-sort TreeSort: n indsættelser $\rightarrow O(\log n) \rightarrow \Theta(n \log n) \rightarrow$ nej. Ubalanceret TreeSort: sorteret input degenererer til en sti, og indsættelse i koster i :

$$\sum_{i=1}^n i = \Theta(n^2)$$

$\rightarrow$ ja. QuickSort og InsertionSort: begge $\Theta(n^2) \rightarrow$ ja.

MCQ juni 2019, Spm. 11

Kør PARTITION($A, 1, 7$) på arrayet $A = [6, 2, 4, 5, 1, 7, 3]$ (indeks 1..7). Hvilken mulighed viser A bagefter? (Standard CLRS Lomuto-partition; pivot er sidste element $A[7] = 3$.)

- (a) $A = [2, 1, 3, 5, 6, 7, 4]$
- (b) $A = [2, 1, 3, 6, 4, 5, 7]$
- (c) $A = [1, 2, 3, 4, 5, 6, 7]$
- (d) $A = [3, 2, 1, 4, 5, 6, 7]$
- (e) $A = [1, 2, 3, 7, 6, 5, 4]$
- (f) $A = [2, 1, 4, 5, 6, 7, 3]$

Svar. (a) $A = [2, 1, 3, 5, 6, 7, 4]$.

Fremgangsmåde. Pivot $x = A[7] = 3$, start $i = 0$. $j = 1$: $6 > 3$, spring over. $j = 2$: $2 \leq 3$, $i = 1$, swap $\rightarrow [2, 6, 4, 5, 1, 7, 3]$. $j = 3, 4$: $4, 5 > 3$, spring over. $j = 5$: $1 \leq 3$, $i = 2$, swap $A[2], A[5] \rightarrow [2, 1, 4, 5, 6, 7, 3]$. $j = 6$: $7 > 3$, spring over. Til sidst swap $A[i + 1] = A[3]$ med $A[7] \rightarrow [2, 1, 3, 5, 6, 7, 4]$.

MCQ juni 2023, Spm. 11

Et array af ni heltal med værdier 0..6: $A = [2, 0, 6, 2, 3, 5, 5, 1, 2]$ (indeks 1..9). Kør COUNTING-SORT($A, 9, 6$) med et array C med syv pladser (0..6). Hvad er summen af de syv heltal i C ved terminering?

- (a) 0
- (b) 6
- (c) 9
- (d) 22
- (e) 28
- (f) 37

Svar. (f) 37.

Fremgangsmåde. Rå tællinger: $C = [1, 1, 3, 1, 0, 2, 1]$ (sum 9). CLRS efterlader C som prefix-summer, $C[i] += C[i - 1]$, så $C = [1, 2, 5, 6, 6, 8, 9]$:

$$1 + 2 + 5 + 6 + 6 + 8 + 9 = 37$$

Fælden: C ender kumulativt, ikke som rå tællinger.

Divide & Conquer

Divide-and-conquer er rekursion som designmetode: del problemet i mindre stykker af samme slags, løs hvert rekursivt og saml svarene. Basistilfældet løses direkte.

De tre trin:

De tre trin

1. **Del op.** Skær inputtet i mindre dele.
2. **Erobr.** Løs hver del med et rekursivt kald.
3. **Kombinér.** Lim delsvarene sammen til det fulde svar.

Hvert kald deler i a delproblemer af størrelse $\frac{n}{b}$ og bruger $f(n)$ arbejde på at dele op og kombinere. Det giver rekursionsligningen:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Den løses med Master Theorem: sammenlign $f(n)$ med $n^{\log_b a}$ og find det dominerende led. Resten af kapitlet er to algoritmer: Strassen og max-sum.

Sådan løser du den

Analysér en divide-and-conquer-algoritme

1. Find de tre trin: hvordan splittes inputtet (del op), hvad er de rekursive kald (erobr), hvad koster det at samle svarene (kombinér).
2. Tæl de rekursive kald a og delproblemets størrelse $\frac{n}{b}$. Ét kald er nok.
3. Mål det ikke-rekursive arbejde $f(n)$ (del op plus kombinér). For matrixarbejde er det blokadditionerne, $\Theta(n^2)$.
4. Opstil rekursionsligningen og løs den med Master Theorem (se kapitlet om rekursionsligninger).

Rekursionstræet

Rekursionstræet er kontrolflowet tegnet ud: hver knude er ét kald, roden er det øverste kald, bladene er basistilfælde. Afviklingen er dybde-først: ned i et barn, bliv færdig, videre til næste. Kaldstakken er stien fra rod til det kald, du er i nu.

Strassen: matrixmultiplikation

Naiv multiplikation af to $n \times n$ -matricer koster $\Theta(n^3)$: hver af de n^2 outputtal er et prikprodukt med n multiplikationer.

Del hver matrix i fire $\frac{n}{2} \times \frac{n}{2}$ -blokke og gang blokvis. Den oplagte opskrift bruger 8 delmultiplikationer plus billige blokadditioner:

$$T(n) = 8T\left(\frac{n}{2}\right) + n^2 = \Theta(n^3)$$

Ingen gevinst: 8 delmultiplikationer er præcis det arbejde, den kubiske metode allerede laver.

Strassen [1969] regner de samme fire outputblokke ud med kun **7** produkter. De ekstra additioner er $\Theta(n^2)$ og betyder intet. Færre rekursive kald flytter eksponenten:

$$T(n) = 7T\left(\frac{n}{2}\right) + n^2 = \Theta(n^{\log_2 7}) = O(n^{2.81})$$

7 mod 8 produkter

Gevinsten er 7 mod 8, fordi $\log_2 7 = 2.807 < 3$. Matrixprodukter er ikke kommutative, så rækkefølgen af faktorerne i hvert produkt P_i skal holdes.

Max-sum: maximum subarray

Find det sammenhængende stykke af et array med den største sum. Det tomme stykke tæller med og har værdien 0, så svaret bliver aldrig negativt; er alle tal negative, vinder det tomme stykke.

Tre algoritmer, hver genbruger det arbejde den forrige smed væk:

Algoritme	Idé	Tid
MaxSum1	Brute force: gensummér hvert stykke fra bunden, tre løkker	$\Theta(n^3)$
MaxSum2	For hvert startpunkt udvides den løbende sum med én addition	$\Theta(n^2)$
MaxSum3 (Kadane)	Bedste stykke der slutter i i udvider det forrige	$\Theta(n)$

Kadane bygger på én observation: det bedste stykke der slutter i position i er enten det bedste der sluttede i $i - 1$ udvidet med $A[i]$, eller det tomme stykke:

$$\text{maxEndingHere}_i = \max(\text{maxEndingHere}_{i-1} + A[i], 0)$$

Det tomme stykke

Nullet er det tomme stykke; det nulstiller den løbende sum, før den når at blive negativ. Algoritmen holder kun styr på to tal, så pladsen er $O(1)$.

Max-produkt via log

Et max-produkt-problem bliver til max-sum med en logaritme. $\log$ er voksende med $\log(xy) = \log x + \log y$, så det bedste produktstykke er det bedste sumstykke af log-værdierne.

```

MaxSum3(n)                                // Kadane, Theta(n)
    maxSoFar = 0
    maxEndingHere = 0
    for i = 0 to n - 1
        maxEndingHere = max(maxEndingHere + A[i], 0)
        maxSoFar = max(maxSoFar, maxEndingHere)
    return maxSoFar

```

Tilbagevendende eksamensspørgsmål

Naiv rekursiv matrixmultiplikation deler hver $n \times n$ -matrix i fire blokke og kalder rekursivt med 8 delmultiplikationer:

$$T(n) = \underline{8}T(n/\underline{2}) + n^2$$

Hvilken Θ -grænse gælder, og slår den den naive $\Theta(n^3)$?

Svar. $T(n) = \Theta(n^3)$. Ingen forbedring.

Fremgangsmåde. $a = \underline{8}$, $b = \underline{2}$, $f(n) = n^2$. Skellet er $\alpha = \log_2 8 = 3$, så

$$n^\alpha = n^3.$$

Her er $f(n) = n^2 = O(n^{3-\varepsilon})$ med $\varepsilon = 1$, polynomielt mindre. Master case 1 giver $T(n) = \Theta(n^3)$, samme som den kubiske metode.

Strassen slides (SE4-DMAD F26)

Strassen regner de fire outputblokke ud med 7 delmultiplikationer i stedet for 8:

$$T(n) = \underline{7}T(n/\underline{2}) + n^2$$

Hvad er køretiden?

Svar. $T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81})$.

Fremgangsmåde. $a = \underline{7}$, $b = \underline{2}$, $f(n) = n^2$. Skellet er $\alpha = \log_2 7 = 2.807\dots$, så

$$n^\alpha \approx n^{2.81}.$$

Her er $f(n) = n^2 = O(n^{\alpha-\varepsilon})$, polynomielt mindre, så Master case 1 giver $T(n) = \Theta(n^\alpha) = O(n^{2.81})$. Det slår $\Theta(n^3)$ fordi $\log_2 7 < 3$.

Max-sum slides (SE4-DMAD F26)

Hvad er køretid og ekstra pladsforbrug for **Kadanes algoritme (MaxSum3)** til maximum subarray-problemet?

Svar. $\Theta(n)$ tid, $O(1)$ plads.

Fremgangsmåde. Ét gennemløb med n iterationer á $\Theta(1)$ arbejde (to `max`-opdateringer) giver $\Theta(n)$. Algoritmen holder kun to tal (`maxSoFar`, `maxEndingHere`) uden ekstra array, så pladsen er $O(1)$. `MaxSum1` er $\Theta(n^3)$ (tre løkker), `MaxSum2` er $\Theta(n^2)$ (to løkker); hvert spring fjerner én løkke ved at genbruge den forrige sum.

MCQ juni 2023, Spm. 2

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = \underline{1}T(n/\underline{2}) + \underline{n^{1/2}}$$

- (a) $T(n) = \Theta(\log n)$
- (b) $T(n) = \Theta(n^{\frac{1}{2}})$
- (c) $T(n) = \Theta(n)$
- (d) $T(n) = \Theta(n \log n)$
- (e) $T(n) = \Theta(n^{\frac{3}{2}})$
- (f) $T(n) = \Theta(n^2)$
- (g) Rekursionsligningen kan ikke løses med Master Theorem.

Svar. (b) $T(n) = \Theta(n^{\frac{1}{2}})$.

Fremgangsmåde. $a = \underline{1}$, $b = \underline{2}$, $f(n) = \underline{n^{\frac{1}{2}}}$. Skellet er $\alpha = \log_2 1 = 0$, så

$$n^\alpha = n^0 = 1.$$

Her er $f(n) = n^{0.5}$ polynomielt større end 1, altså $f(n) = \Omega(n^{0+\epsilon})$, så Master case 3. Tjek regularitet:

$$a f\left(\frac{n}{b}\right) = \sqrt{\frac{n}{2}} = \frac{\sqrt{n}}{\sqrt{2}} \leq c \sqrt{n}, \quad c = \frac{1}{\sqrt{2}} < 1.$$

Den holder, så $T(n) = \Theta(f(n)) = \Theta(n^{\frac{1}{2}})$.

Dynamisk programmering

Når et problem deles op i mindre udgaver af sig selv, og udgaverne overlapper, regner naiv rekursion det samme svar igen og igen — eksponentielt dyrt. Dynamisk programmering løser hvert delproblem én gang og gemmer svaret i en tabel. Du skriver den optimale værdi som en rekursion og fylder tabellen, så hver celle kun beregnes én gang.

Til eksamen får du typisk en færdig rekursion og skal angive køretid eller mindste plads. Sjældnere skal du selv opstille rekursionen, fylde tabellen og rekonstruere løsningen.

Sådan løser du den

To tal aflæses direkte af rekursionen, uden at køre den.

Køretiden er antal celler gange arbejdet per celle:

$$T = (\text{antal celler}) \times (\text{arbejde per celle})$$

Mindste plads er den største mængde tidligere celler, du er nødt til at holde på samtidig:

$$S = \text{størrelsen af den front, du skal huske}$$

Læs køretid og plads af en given rekursion

1. Find tabellens størrelse ud fra indeksenens intervaller. i i $0..m$ og j i $0..n$ giver $\Theta(mn)$ celler. Et par $i \leq j$ giver en trekant på $n(n+1)/2 = \Theta(n^2)$ celler.
2. Find arbejdet per celle. Et min eller max over op til j tidligere celler koster $\Theta(j)$; to opslag plus aritmetik koster $\Theta(1)$.
3. Gang de to sammen.
4. For plads: se hvilke tidligere celler en celle læser. Kun den forrige række $\rightarrow$ én række er nok. Alle tidligere celler $\rightarrow$ intet kan smides væk.

Byg en DP-løsning fra bunden

1. Vælg delproblemets størrelse som ét eller flere heltalsindeks. Det fastlægger tabellens dimensioner.
2. Argumentér for optimale delproblemer: skær en optimal løsning i en sidste del og resten, og vis at resten selv er optimal for et mindre delproblem.
3. Skriv rekursionen som et max eller min over alle valg af den sidste del, plus et basistilfælde for størrelse 0.
4. Fyld tabellen bottom-up, så hver celledes afhængigheder allerede står klar. Eller brug memoization: rekursér top-down og cache hver celle første gang.
5. Vil du have selve løsningen, så gem det vindende valg per celle og følg valgene baglæns.

Memoization vs. bottom-up

Memoization og bottom-up giver samme Θ for tid og plads; memoization har blot en lidt dårligere konstant.

Problem	Celler	Arbejde/celle	Tid
Stangskæring $r(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$
LCS $\text{lcs}(i, j)$	$\Theta(mn)$	$\Theta(1)$	$\Theta(mn)$
Matrixkæde $m(i, j)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n^3)$

Arbejde per celle

Tabellens størrelse er ikke køretiden — gang med arbejdet per celle. Et min over op til j tidligere celler koster $\Theta(j)$, ikke $\Theta(1)$, og gør en $\Theta(mn)$ -tabel til $\Theta(mn^2)$.

Rullende array

Mindste plads er ikke altid tabellens størrelse. Afhænger en celle kun af den forrige række, nøjes et rullende array med én række, og pladsen falder fra $\Theta(mn)$ til $\Theta(n)$. Læser en celle alle tidligere indgange, kan intet frigives.

Rekonstruktion

Tabellen rummer kun den optimale værdi. Vil du have løsningen, så gem det vindende valg i hver celle og gå baglæns fra målcellen til basistilfældet. Det koster $O(n)$ eller $O(m + n)$ ekstra — langt mindre end fyldningen.

Tilbagevendende eksamensspørgsmål

MCQ juni 2017, Spm. 25

For et optimeringsproblem er inputtet en omkostning c_{ij} for alle i og j . En løsning $b(i, j)$ kan beskrives ved rekursionen

$$b(i, j) = \begin{cases} 0 & \text{hvis } i = 0 \text{ eller } j = 0 \\ c_{ij} + \min\{b(i-1, k) : 0 \leq k < j\} & \text{hvis } i > 0 \text{ og } j > 0 \end{cases}$$

Hvis $b(\underline{m}, \underline{n})$ findes ved dynamisk programmering, hvilken køretid opnås da?

- (a) $\Theta(1)$
- (b) $\Theta(m)$
- (c) $\Theta(n)$
- (d) $\Theta(mn)$
- (e) $\Theta(m^2n)$
- (f) $\Theta(mn^2)$
- (g) $\Theta(m^2n^2)$

Svar. (f) $\Theta(mn^2)$.

Fremgangsmåde. Celler: i i $0..m$, j i $0..n$ giver $\Theta(mn)$. Per celle løber min'en over $k \in \{0, \dots, j-1\}$, op til $j = \Theta(n)$ led. Samlet

$$\Theta(mn) \cdot \Theta(n) = \Theta(mn^2)$$

MCQ juni 2017, Spm. 26

Samme problem og samme rekursion for $b(i, j)$. Hvis $b(\underline{m}, \underline{n})$ findes ved dynamisk programmering, hvad er det mindste pladsforbrug, der kan opnås?

- (a) $\Theta(1)$
- (b) $\Theta(m)$
- (c) $\Theta(n)$
- (d) $\Theta(mn)$
- (e) $\Theta(m^2n)$
- (f) $\Theta(mn^2)$
- (g) $\Theta(m^2n^2)$

Svar. (c) $\Theta(n)$.

Fremgangsmåde. Hver celle i række i læser kun række $i - 1$. Et rullende array på én række er nok: smid række $i - 1$ væk, når række i er færdig. Lavere går ikke, for en celle kræver et præfiks af alle n søjler i forrige række. Mindste plads = $\Theta(n)$, ikke den fulde $\Theta(mn)$ -tabel.

MCQ juni 2025, Spm. 30

Find $B(n)$, antallet af binære træer med n knuder (fx $B(3) = 5$), ved rekursionen

$$B(0) = 1, \quad B(n) = \sum_{i=0}^{n-1} B(i) \cdot B(n-i-1) \text{ for } n > 0$$

Beregnet ved dynamisk programmering på denne rekursion, hvilken køretid opnås?

- (a) $\Theta(1)$
- (b) $\Theta(\log n)$
- (c) $\Theta(n)$
- (d) $\Theta(n \log n)$
- (e) $\Theta(n^2)$
- (f) $\Theta(n^3)$

Svar. (e) $\Theta(n^2)$.

Fremgangsmåde. 1-D-tabel $B[0..n]$, altså $\Theta(n)$ celler. $B[k]$ summerer k produkter, så $\Theta(k)$ arbejde. Samlet

$$\sum_{k=1}^n k = \frac{n(n+1)}{2} = \Theta(n^2)$$

Pladsen er $\Theta(n)$: $B(n)$ læser hver tidligere indgang, så ingen kan frigives. Et konstant vindue som i Fibonacci dur ikke her.

MCQ juni 2019, Spm. 29

For et optimeringsproblem er inputtet værdier c_k for $k = 1, 2, \dots, n$. En løsning $L(i, j)$ kan beskrives ved rekursionen

$$L(i, j) = \begin{cases} c_i & \text{hvis } i = j \\ i \cdot L(i+1, j) + j \cdot L(i, j-1) & \text{hvis } i < j \end{cases}$$

Hvis $L(1, n)$ findes ved dynamisk programmering, hvilken køretid opnås?

- (a) $\Theta(1)$
- (b) $\Theta(n^{1/2})$
- (c) $\Theta(n)$
- (d) $\Theta(n^{3/2})$
- (e) $\Theta(n^2)$
- (f) $\Theta(n^3)$

Svar. (e) $\Theta(n^2)$.

Fremgangsmåde. Gyldige tilstande er par $1 \leq i \leq j \leq n$, en øvre trekant på $n(n+1)/2 = \Theta(n^2)$ celler. Hver celle med $i < j$ læser to værdier plus $O(1)$ aritmetik, så $\Theta(1)$ arbejde. Samlet

$$\Theta(n^2) \cdot \Theta(1) = \Theta(n^2)$$

Mindste plads er $\Theta(n)$: ordn efter diagonalen $d = j - i$, hvor diagonal d kun kræver diagonal $d - 1$.

Greedy-algoritmer

En greedy-algoritme bygger løsningen op ét skridt ad gangen og tager hver gang det, der ser bedst ud lige nu, uden at fortryde. Hurtigt, men kun korrekt hvis det lokale valg beviseligt giver den globalt bedste løsning.

Til eksamen kommer det to steder fra. Huffman-kodning (næsten hvert år): byg et træ ud fra symbolfrekvenser og aflæs en kodeordslængde eller det samlede antal bit. Og argumentet for, **hvorfor** et greedy-valg er optimalt — typisk et ombytningsargument.

Sådan løser du den

Huffman giver hyppige symboler korte koder og sjældne symboler lange. Et symbols kodeordslængde er dybden af dets blad: jo dybere, jo flere bit.

Byg Huffman-træet

1. Skriv symbolerne op med deres frekvenser. Der er n af dem.
2. Læg alle bladene i en min-prioritetskø med frekvensen som nøgle.
3. Tag de to mindste vægte x og y ud, lav en knude med vægt $x + y$, og læg den tilbage. Notér $x + y$; det er den interne knudes vægt.
4. Gentag til der er ét træ tilbage — $n - 1$ merges.
5. Kodeordslængden for et symbol er dybden af dets blad.

For et symbol s med frekvens $\text{freq}(s)$ og bladdybde $\text{depth}(s)$ er det samlede antal bit:

$$\sum_s \text{freq}(s) \cdot \text{depth}(s)$$

Den sum er lig summen af alle interne knuders vægte, du skrev op undervejs:

$$\sum_s \text{freq}(s) \cdot \text{depth}(s) = \sum_{\text{interne knuder}} \text{vægt}$$

Interne vægte

Genvejen via interne vægte er hurtigst: læg blot de noterede merge-summer sammen. For o:150, p:100, q:25, r:125, s:200, t:50, u:75 bliver de interne vægte 75, 150, 225, 300, 425, 725, og $75 + 150 + 225 + 300 + 425 + 725 = 1900$ bit. Slip for at holde styr på dybder.

Flere gyldige træer

Ved samme vægt kan begge træer vælges først, og venstre/højre er frit, så der findes flere gyldige Huffman-træer. Men dybder og pris er ens uanset valget — svar trygt på dem. Kun “tegn træet” har flere rigtige svar.

Optimal vs. producerbar

Optimal betyder ikke, at Huffman kan bygge det. Huffman merger altid de to aktuelt mindste vægte, så et producerbart træ må have interne vægte, der passer med den tvungne merge-rækkefølge. To optimale træer kan have hvert sit sæt interne vægte; kun dem der matcher Huffmans merges er gyldige.

Aktivitetssudvælgelse er det andet klassiske greedy-problem: du har aktiviteter med start- og sluttider og vil have flest mulige uden overlap.

Aktivitetssudvælgelse

1. Sortér de n aktiviteter efter voksende sluttidspunkt.
2. Gå dem igennem og vælg en aktivitet hvis den ikke overlapper den senest valgte; ellers spring den over.
3. Resultatet er en størst mulig mængde uden overlap.

Køretiden er domineret af sorteringen:

$$O(n \log n)$$

Korrekt greedy-valg

De andre greedy-valg fejler: korteste aktivitet, færrest overlap og tidligste start har alle modeksempler. Kun tidligste sluttidspunkt er optimalt. Beviset er en invariant — “en optimal løsning indeholder de hidtidige valg” — lukket med et ombytningsargument.

0-1-rygsæk

Greedy efter værditæthed løser fractional rygsæk optimalt, men fejler for 0-1. Kapacitet 50, genstande (vægt 10, \$60), (vægt 20, \$100), (vægt 30, \$120): tæthed tager de to første og får \$160, men optimum er de to sidste med \$220. 0-1-rygsæk kræver dynamisk programmering.

Skal du **bevise** greedy optimal, brug et ombytningsargument: tag en vilkårlig optimal løsning OPT og vis, at du kan bytte ét element i OPT ud med greedy-valget uden at gøre OPT værre eller ugyldig. Gentaget bliver OPT til greedy-løsningen, så greedy er også optimal.

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 13 (samme skabelon juni 2019/2021/2023)

En fil indeholder tegn med hyppigheder: o=150, p=100, q=25, r=125, s=200, t=50, u=75. Byg et Huffman-træ. Hvor mange bit er der i kodeordet for q ?

- (a) 1
- (b) 2
- (c) 3
- (d) 4
- (e) 5
- (f) 6

Svar. (d) 4.

Fremgangsmåde. Sortér: q:25, t:50, u:75, p:100, r:125, o:150, s:200. Merges: $q + t = 75$; $u + 75 = 150$; $p + r = 225$; $o + 150 = 300$; $s + 225 = 425$; $300 + 425 = 725$ (rod). q ligger under $(q + t)$, $(u + 75)$, $(o + 150)$ og roden: dybde 4, altså 4 bit.

MCQ juni 2025, Spm. 14

Et Huffman-træ for en fil med hyppigheder $o=150$, $p=100$, $q=25$, $r=125$, $s=200$, $t=50$, $u=75$ (i alt 725 symboler). Hvor mange bit fylder de 725 symboler tilsammen Huffman-kodet?

- (a) 1800
- (b) 1825
- (c) 1900
- (d) 1925
- (e) 2000
- (f) 2075
- (g) 2100

Svar. (c) 1900.

Fremgangsmåde. Genvej via interne vægte: merges gav 75, 150, 225, 300, 425, 725, og $75 + 150 + 225 + 300 + 425 + 725 = 1900$. Tjek med $\sum \text{freq} \cdot \text{depth}$: $150 \cdot 2 + 200 \cdot 2 + 100 \cdot 3 + 125 \cdot 3 + 75 \cdot 3 + 25 \cdot 4 + 50 \cdot 4 = 1900$.

MCQ juni 2021, Spm. 12

En fil indeholder tegnene med hyppigheder: $a=200$, $b=250$, $c=100$, $d=350$, $e=400$. Byg et Huffman-træ. Hvor mange bit er der i kodeordet for d ?

- (a) 1
- (b) 2
- (c) 3
- (d) 4

Svar. (b) 2.

Fremgangsmåde. Merges: $c + a = 300$; $b + d = 600$; $300 + e = 700$; $600 + 700 = 1300$ (rod). d er barn af $(b + d)$, som er barn af roden: dybde 2, altså 2 bit.

DM507 juni 2012, Opg. 5c

Frekvenser $a=100$, $b=150$, $c=150$, $d=250$, $e=350$ (i alt 1000). Alle de viste træer er optimale (pris 2250 bit). Hvilke af træerne H2–H5 kan Huffman faktisk producere?

- (a) $H2 = ((c, d), ((a, b), e))$
- (b) $H3 = ((b, e), ((a, c), d))$
- (c) $H4 = ((d, (a, b)), (c, e))$
- (d) $H5 = ((b, (a, c)), (d, e))$

Svar. (a) og (d): kun H2 og H5.

Fremgangsmåde. Huffmans tvungne merges: $100 + 150 = 250$; $150 + 250 = 400$; $250 + 350 = 600$; $400 + 600 = 1000$. Et producerbart træ må have interne vægte 250, 400, 600, 1000. H2 og H5 giver

begge netop dem og matcher. H3 og H4 giver 250, 500, 500, 1000; de to 500-taller er parringer Huffman aldrig laver — optimale, men ikke producerbare.

Prioritetskøer og heaps

En prioritetskø trækker hele tiden den vigtigste nøgle ud — den mindste i en min-kø, den største i en max-kø. Den implementeres typisk som en binær heap.

En binær heap er et komplet binært træ gemt i et array. “Komplet” betyder, at træet fyldes lag for lag fra venstre uden huller. Hver forælder dominerer sine børn: i en min-heap er forælderen aldrig større, i en max-heap aldrig mindre.

Træet ligger i et 1-indeksret array $A[1..n]$, og indekset alene giver børn og forælder:

$$\text{forælder}(i) = \lfloor i/2 \rfloor, \quad \text{venstre}(i) = 2i, \quad \text{højre}(i) = 2i + 1$$

Et array **er** et træ. Tag $A = [18, 9, 16, 4, 8, 12, 13, 1, 2]$ — sådan ser det ud med pladsnumrene over:

	1	2	3	4	5	6	7	8	9
A:	18	9	16	4	8	12	13	1	2

Plads 1 er roden. Børnene af plads i står på $2i$ og $2i + 1$, forælderen på $\lfloor i/2 \rfloor$, så plads 1 har børn på 2 og 3, plads 2 har børn på 4 og 5, og så videre. Hver forælder er $\geq$ sine børn (max-heap), så den største (18) ligger forrest. **Boble op** flytter en værdi mod plads 1, **synk ned** flytter den mod de bagerste pladser. Det er alt operationerne gør.

Til eksamen skal du afgøre om et array er en heap, og trace **Extract**, **Increase-Key** og **Insert** skridt for skridt.

Sådan løser du den

Hver operation har sin egen opskrift herunder. De er skrevet for en **max-heap** (største øverst). Kører du en **min-heap**, byt blot “størst/større” ud med “mindst/mindre” overalt. Husk positions-reglerne: barn af plads i ligger på $2i$ og $2i + 1$; forælder ligger på $\lfloor i/2 \rfloor$.

Heap-Increase-Key(A, i, k) — gør en nøgle større

Hvad det gør: sætter nøglen på plads i op til en større værdi k , og lader den “boble opad” til sin rette plads.

Increase-Key — boble op

1. Skriv den nye værdi ind: $A[i] = k$. (I en max-heap skal k være $\geq$ den gamle værdi.)
2. Find forælderen på plads $\lfloor i/2 \rfloor$.
3. Er din nøgle **større** end forælderen, så **byt** de to. Er den ikke, **stop**; så er du færdig.
4. Efter byttet står din nøgle på forælders gamle plads. Gentag fra trin 2 derfra, indtil forælderen er større, eller du rammer roden (plads 1).

Heap-Extract-Max(A) — træk den største ud

Hvad det gør: roden (plads 1) er altid den største. Du fjerner den, sætter en ny værdi i roden og lader den “synke nedad”, til heap-ordenen passer igen.

Extract-Max — synk ned

1. Roden $A[1]$ er svaret (den største). Den fjernes.
2. Flyt det **sidste** element op på plads 1. Heapen bliver nu én plads kortere (den sidste plads findes ikke længere).
3. Kig på plads 1's to børn (plads 2 og 3). Find den **største** af de tre: forælder, venstre barn, højre barn.
4. Er et **barn** størst, **byt** forælder med det barn. Er **forælderen** størst, **stop**.
5. Efter byttet står din nøgle på barnets plads. Gentag fra trin 3 derfra, indtil intet barn er større, eller knuden ingen børn har. Aflæs arrayet.

Insert(A, k) — indsæt en ny nøgle

Hvad det gør: lægger en ny nøgle ind og bobler den op, præcis som Increase-Key.

Insert — læg på og boble op

1. Læg den nye nøgle på første ledige plads (lige efter det sidste element). Træet er stadig komplet.
2. Boble op nøjagtigt som i Increase-Key: byt med forælderen så længe din nøgle er større, stop ellers.

Tjek om et array er en heap

Er det en gyldig heap?

1. Tjek for huller: står der en værdi efter en tom plads, er arrayet ikke en gyldig heap.
2. Løb de interne knuder igennem, $i = 1..[n/2]$, for kun de har børn.
3. Tjek begge børn for hver knude. Min-heap kræver $A[i] \leq A[2i]$ og $A[i] \leq A[2i + 1]$; max-heap kræver $A[i] \geq$ begge børn. Ens nøgler er tilladt.
4. Én overtrådt ulighed er nok til at forkaste arrayet.

Boble op vs. synk ned

Den klassiske trace-fejl er at bytte retning om. Increase-Key og Insert **bobler op** (mod forælderen, $[i/2]$). Extract **synker ned** (mod børnene, $2i$ og $2i + 1$). Roden er min i en min-heap og max i en max-heap, aldrig begge.

Build-Heap-køretid

Build-Heap er $O(n)$, ikke $O(n \log n)$. Den kalder synk-ned nedefra og op over arrayets nederste halvdel. At indsætte n nøgler én ad gangen ville koste $O(n \log n)$.

Tilbagevendende eksamensspørgsmål

DM507 juni 2012, Opg. 2a (5%)

Hvilke af følgende arrays af størrelse 10 er min-heaps?

$A1 = [7, 4, 9, 2, 6, 8, 10, 1, 3, 5]$

$A2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

$A3 = [1, 2, 3, 4, 1, 2, 3, 4, 5, 6]$

$A4 = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]$

Svar. A2 og A4.

Fremgangsmåde. Tjek $A[i] \leq A[2i]$ og $A[i] \leq A[2i+1]$ for $i = 1..5$. A1: $A[1] = 7 > A[2] = 4$, falder. A2: sorteret stigende, forælder $\leq$ børn — min-heap. A3: $A[2] = 2 > A[5] = 1$, falder. A4: alle ens — min-heap.

MCQ juni 2023, Spm. 7

Udfør Extract-Min på min-heapen $A = [3, 5, 6, 10, 11, 8, 7, 18, 16, 15]$ (1-indeksret, positioner 1..10). Hvilken position i A ender 15 på bagefter?

- (a) 1
- (b) 2
- (c) 3
- (d) 4
- (e) 5
- (f) 8
- (g) 9

Svar. (d) position 4.

Fremgangsmåde. Fjern roden 3, flyt sidste element 15 op, størrelse 9: $[15, 5, 6, 10, 11, 8, 7, 18, 16]$. Sift ned fra idx 1: børn 5, 6, byt med 5 — 15 til idx 2. Idx 2: børn 10, 11, byt med 10 — 15 til idx 4. Børn ved idx 8 (18) og 9 (16) er begge > 15 , stop. Slut $[5, 10, 6, 15, 11, 8, 7, 18, 16]$: 15 står på position 4.

MCQ juni 2015, Spm. 6

Udfør først Heap-Increase-Key($A, 9, 15$) og derefter Heap-Extract-Max(A) på max-heapen A herunder. A (positioner 1..9) = $[18, 9, 16, 4, 8, 12, 13, 1, 2]$. Hvilket af svarene viser heapen efter de to operationer?

- (a) $A = [16, 15, 13, 9, 8, 4, 12, 1]$
- (b) $A = [16, 15, 13, 9, 8, 12, 4, 1]$
- (c) $A = [16, 15, 13, 4, 8, 12, 2, 1]$
- (d) $A = [16, 15, 13, 4, 8, 12, 1, 2]$

Svar. (b) $[16, 15, 13, 9, 8, 12, 4, 1]$.

Fremgangsmåde. Increase-Key pos 9 til 15: forælder pos 4 = 4 $<$ 15 byt, pos 2 = 9 $<$ 15 byt, pos 1 = 18 $>$ 15 stop — $[18, 15, 16, 9, 8, 12, 13, 1, 4]$. Extract-Max: max = 18, flyt sidste 4 op, størrelse 8, sift ned: mod 15, 16 byt med 16, mod 12, 13 byt med 13, mod barn 1 stop. Slut $[16, 15, 13, 9, 8, 12, 4, 1]$.

MCQ juni 2019, Spm. 28

Hvad er worst-case køretiden for Heapsort, når den køres på n identiske elementer?

- (a) $O(1)$
- (b) $O(\log n)$
- (c) $O(n)$
- (d) $O(n \log n)$
- (e) $O(n^2)$

Svar. (c) $O(n)$.

Fremgangsmåde. Heapsort er Build-Max-Heap ($O(n)$) og derefter $n - 1$ Extract-Max med sift-ned. Når alle nøgler er ens, er en forælder aldrig strengt mindre end et barn, så hvert sift-ned stopper efter et par sammenligninger — $O(1)$ per kald. Build: $\sim n/2$ kald, Extract: $n - 1$ kald, hver $O(1)$. I alt $O(n)$.

Søgestrukturer: BST, augmenteret BST, hashing

En ordbog skal kunne slå op, indsætte og slette. Skal den også svare på “hvad kommer før/efter denne nøgle” eller gå igennem alt i sorteret orden, kræver det en **ordnet** ordbog.

Et balanceret søgetræ (rød-sort træ) klarer alt i $O(\log n)$. En hashtabel klarer opslag, indsæt og slet i $O(1)$ i gennemsnit, men kan ikke svare på de ordnede spørgsmål.

At **augmentere** et BST betyder, at hver knude gemmer ekstra info om hele sit deltræ (antal knuder, største y -værdi, en sum), så du kan svare på nye spørgsmål i $O(\log n)$.

Til eksamen skal du opstille ligningerne for et augmenteret felt, simulere en hashtabel skridt for skridt og udvælge worst-case-køretider.

Sådan løser du den

Augmentér et BST med deltræs-info

1. Beslut hvad hver knude gemmer om **hele sit deltræ**, fx $v.size$ = antal knuder eller $v.ymax$ = største y -værdi.
2. Skriv ligningen for feltet ud fra de to børn og knuden selv. For størrelse:
3. Tjek augmenteringsbetingelsen: kan feltet beregnes i $O(1)$ ud fra børnenes felter, kan det vedligeholdes under indsæt og slet uden ekstra køretid. Det gælder sum, antal, min og max.
4. En forespørgsel går fra roden ét niveau ad gangen og bruger felterne. Det er $O(\log n)$, da træet er balanceret.

$$v.size = 1 + v.left.size + v.right.size$$

For et aggregat over data, som største y -værdi i deltræet:

$$v.ymax = \max(v.y, v.left.ymax, v.right.ymax)$$

Nøglefelt vs. ikke-nøglefelt

Nøglefelt vs. ikke-nøglefelt afgør formen. Træet er sorteret efter **én** nøgle, fx x . Et ekstremum af **nøglen** ligger fast: største x er den højreste knude, mindste x den venstre.

$$v.xmax = v.right.xmax, \quad v.xmin = v.left.xmin$$

Et ikke-nøglefelt (fx y) er ikke sorteret af træet, så kig på begge børn og knuden selv:

$$v.ymax = \max(v.y, v.left.ymax, v.right.ymax)$$

Vedligeholdelse af augmentering

Vedligeholdelse rører ikke kun bladet eller kun roden. En indsæt eller slet ændrer knuderne langs én rod-til-blad-sti plus $O(1)$ rotationer; du genberegner felterne nedefra og op langs den sti, altså $O(\log n)$ knuder. Et fuldt inorder-gennemløb er $O(n)$ og dermed forkert svar på et $O(\log n)$ -spørgsmål.

For ordensstatistik gemmer hver knude `size`. NIL er en sentinel med `size = 0`, så `x.left.size` altid er defineret. Rangen i eget deltræ:

$$r = x.\text{left.size} + 1$$

```

OS-SELECT(x, i)           // knuden med rang i i x's deltræ
  r = x.left.size + 1
  if i == r      return x
  elseif i < r   return OS-SELECT(x.left, i)
  else          return OS-SELECT(x.right, i - r)

```

```

OS-RANK(T, x)             // rang af x i hele træet
  r = x.left.size + 1
  y = x
  while y != T.root
    if y == y.p.right
      r = r + y.p.left.size + 1
    y = y.p
  return r

```

Simulér en hashtabel med open addressing

1. Noter tabelstørrelsen m og hvilke pladser der er optaget før indsættelsen.
2. Find probe-sekvensen. Linear probing:
3. Indsæt ved at prøve $i = 0, 1, 2, \dots$ til **første tomme plads**. Søgning gør det samme og stopper ved nøglen eller en tom plads.
4. Til “hvilke h' -værdier er mulige”: prøv hver kandidat, simulér probingen, og behold den hvis landingen rammer den observerede plads.

$$h(k, i) = (h'(k) + i) \bmod m$$

Double hashing bruger en anden funktion til skridtlængden:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Chaining

Chaining er den anden kollisionsmetode: hver plads peger på en liste over de nøgler, der hasher dertil. Indsæt er $O(1)$ (forrest i listen), søg og slet er $\Theta(|\text{liste}|)$. Worst case hasher alle n nøgler til samme plads, så listen får længde n og søgning bliver $\Theta(n)$. Bruger du balancerede træer i stedet for lister, falder worst case til $O(\log n)$. I praksis er hashing $O(1)$ for søg, indsæt og slet.

Open addressing

Open addressing kræver $n \leq m$; sigt efter en fyldningsgrad omkring $n \approx m/4$. Slet markerer pladsen som slettet uden at tømme den, ellers afbryder en senere søgning for tidligt. Vælg m som primtal, så probe-sekvensen når alle pladser. Quadratic probing er ude af pensum.

Valget mellem de to strukturer afgøres af spørgsmåletypen:

Operation	Balanceret BST	Hashing (gns. / worst)
Søg / indsæt / slet	$O(\log n)$	$O(1)$ / $\Theta(n)$
Predecessor / successor	$O(\log n)$	ikke understøttet
Sorteret gennemløb	$O(n)$	ikke understøttet

Rød-sort balance

Et rød-sort træ er balanceret, fordi ingen sti har to røde knuder i træk. Med k sorte knuder på hver rod-til-blad-sti gælder $n \geq 2^{k-1} - 1$, så $k - 1 \leq \log(n + 1)$. Den længste sti har højst dobbelt så mange kanter som den korteste, så højden er $\leq 2 \log(n + 1) = O(\log n)$.

Tilbagevendende eksamensspørgsmål

MCQ juni 2015, Spm. 25

En mængde punkter i planen gemmes i et balanceret binaert søgetræ (BST) med punkternes x -koordinat som nøgle. Hver knude svarer til et punkt og gemmer derudover fire felter for hele sit deltræ: $v.xmax$, $v.xmin$, $v.ymax$, $v.ymin$. Hvilket sæt opdateringsligninger vedligeholder felterne korrekt i knude v ud fra dens børn $v.left$, $v.right$ og dens egne koordinater $v.x$, $v.y$?

- (a) $v.xmax = v.left.xmax$; $v.xmin = v.right.xmin$; $v.ymax = \max(v.left.ymax, v.right.ymax, v.y)$; $v.ymin = \min(v.left.ymin, v.right.ymin, v.y)$
- (b) $v.xmax = v.right.xmax$; $v.xmin = v.left.xmin$; $v.ymax = \min(v.left.ymax, v.right.ymax, v.y)$; $v.ymin = \max(v.left.ymin, v.right.ymin, v.y)$
- (c) $v.ymax = v.right.ymax$; $v.ymin = v.left.ymin$; $v.xmax = \max(v.left.xmax, v.right.xmax, v.x)$; $v.xmin = \min(v.left.xmin, v.right.xmin, v.x)$
- (d) $v.xmax = v.right.xmax$; $v.xmin = v.left.xmin$; $v.ymax = \max(v.left.ymax, v.right.ymax, v.y)$; $v.ymin = \min(v.left.ymin, v.right.ymin, v.y)$

Svar. Mulighed (d).

Fremgangsmåde. Træet er sorteret efter x , så $xmax$ kommer fra højre barn og $xmin$ fra venstre. y er ikke ordnet af træet, så $ymax$ og $ymin$ kombineres over begge børn og knuden selv. (a) bytter x -siderne, (b) bytter max og min på y , (c) henter y fra ét fast barn. Kun (d) er rigtig.

MCQ juni 2015, Spm. 26

Lad nu søgetræet være et rød-sort træ. Hvilke af følgende er korrekte argumenter for, at informationen i træets knuder kan vedligeholdes under indsættelser og sletninger, uden at ændre de rød-sortede træers køretid på $O(\log n)$ for indsættelser og sletninger?

- (a) Ingen information i knuderne behøver at ændres under indsættelser og sletninger, så de rød-sortede træers $O(\log n)$ -køretid holder stadig.
- (b) Kun roden får ny information, og det er $O(1)$ ekstra arbejde oven i de rød-sortede træers $O(\log n)$ -køretid, hvilket stadig er $O(\log n)$.
- (c) Ved indsættelser behøver kun det nye blad ny information ($O(1)$ ekstra), og ved sletninger kan information kun forsvinde, hvilket ikke giver arbejde.
- (d) Efter ændringer i træet under indsættelser og sletninger behøver kun knuder på en sti fra et blad til roden ny information, og den kan genberegnes nedefra og op i tid $O(\log n)$.
- (e) Efter ændringer kan et inorder-gennemløb af træet genoprette informationen i alle knuder i tid $O(\log n)$.

Svar. Mulighed (d).

Fremgangsmåde. En indsæt eller slet ændrer kun knuder på én rod-til-blad-sti plus $O(1)$ rotationer, og hver rotation ændrer lokalt $O(1)$ felter, der kan genfindes fra børnene. Genberegning nedefra og op koster $O(\log n) = \text{træets højde}$. (a) falsk: info ændres. (b) falsk: mere end roden. (c) falsk: sletninger kræver også opdatering. (e) falsk: inorder er $O(n)$.

MCQ juni 2021, Spm. 9 (samme type 2015/2017/2019/2023)

En hashtabel H bruger linear probing og en hjælpe-hashfunktion $h'(x)$. Tabellen er nu (indeks 0..6): plads 0 = 12, plads 1 tom, plads 2 = 10, plads 3 tom, plads 4 = 22, plads 5 tom, plads 6 = 31. Derefter indsættes 97, og tabellen er bagefter: plads 0 = 12, plads 1 = 97, plads 2 = 10, plads 3 tom, plads 4 = 22, plads 5 tom, plads 6 = 31. Tabelstørrelse $m = 7$. Hvilke værdier af $h'(97)$ er mulige? (et eller flere svar)

- (a) $h'(97) = 0$
- (b) $h'(97) = 1$
- (c) $h'(97) = 2$
- (d) $h'(97) = 3$
- (e) $h'(97) = 4$
- (f) $h'(97) = 5$
- (g) $h'(97) = 6$

Svar. Mulighederne (a), (b) og (g): $h'(97) \in \{0, 1, 6\}$.

Fremgangsmåde. Optagne pladser: $\{0, 2, 4, 6\}$, tomme: $\{1, 3, 5\}$, og 97 landede i plads 1. Prob hver startværdi til første tomme plads. $h' = 0$: 0 optaget, lander 1. $h' = 1$: lander 1. $h' = 2 \rightarrow 3$. $h' = 3 \rightarrow 3$. $h' = 4 \rightarrow 5$. $h' = 5 \rightarrow 5$. $h' = 6$: 6 og 0 optaget, lander 1. Kun 0, 1, 6 rammer plads 1.

MCQ juni 2019, Spm. 27 (worst-case sortering, går igen bredt)

Vi betragter sortering af n heltal med værdier i intervallet $[0, n^2)$. Med TreeSort menes algoritmen, der indsætter tallene ét ad gangen i et søgetræ og derefter laver et inorder-gennemløb. Hvilke af følgende algoritmer har worst-case-køretid $\Theta(n^2)$? (et eller flere svar)

- (a) CountingSort
- (b) RadixSort, hvor heltallene behandles som bestående af to cifre med værdier i intervallet $[0, n)$.
- (c) MergeSort
- (d) TreeSort, hvor det binære søgetræ er et rød-sort træ.
- (e) TreeSort, hvor det binære søgetræ er et ubalanceret søgetræ.
- (f) QuickSort
- (g) InsertionSort

Svar. Mulighederne (a), (e), (f) og (g): CountingSort, ubalanceret TreeSort, QuickSort og InsertionSort.

Fremgangsmåde. CountingSort med univers $k = n^2$ er $\Theta(n + n^2) = \Theta(n^2)$. RadixSort med 2 cifre i base n er to CountingSort-pas med $k = n$, altså $\Theta(n)$. MergeSort er altid $\Theta(n \log n)$. TreeSort

på et rød-sort træ er n indsættelser à $O(\log n)$ plus et $O(n)$ -gennemløb, altså $\Theta(n \log n)$. TreeSort på et ubalanceret BST degenererer ved sorteret input til en sti, hvor indsættelse i koster i , så summen er $\Theta(n^2)$. QuickSort og InsertionSort er $\Theta(n^2)$ i worst case.

Disjunkte mængder (Union-Find)

n elementer starter hver for sig. Du slår nogle sammen i grupper og vil løbende kunne spørge: “ligger de to i samme gruppe?” Det løser en disjunkt-mængde-struktur.

Tre operationer: **Make-Set**(x) laver en ny gruppe med kun x . **Find-Set**(x) returnerer repræsentanten for x 's gruppe. **Union**(x, y) smelter to grupper sammen.

Hver gruppe er et træ med repræsentanten som rod, så hele strukturen er en skov. Til eksamen skal du kunne tegne skoven efter en række operationer og kende priserne.

Den dukker oftest op i Kruskals MST-algoritme: for hver kant spørger **Find-Set**, om endepunkterne allerede ligger i samme komponent.

Sådan løser du den

To tricks holder træerne flade. **Union by rank** hænger det lave træ under det høje. **Path compression** peger alle knuder på en **Find-Set**-sti direkte op på roden.

Hver rod har en rank, en øvre grænse for højden. Et frisk **Make-Set** giver

$$x.p = x, \quad x.rank = 0.$$

Link-reglen sammenligner de to rødders rank:

$$\text{rank}(r_x) > \text{rank}(r_y) \implies r_y.p = r_x \quad (\text{rank uændret}).$$

Er rankene lige, taber det **første** argument:

$$\text{rank}(r_x) = \text{rank}(r_y) \implies r_x.p = r_y, \quad r_y.rank + 1.$$

Tegn skoven (union by rank + path compression)

1. **Start.** Hvert **Make-Set** giver en rod med rank 0.
2. **Hver Union**(x, y) er **Link**(**Find-Set**(x), **Find-Set**(y)). Find først begge rødder (her sker path compression), link så.
3. **Link.** Højeste rank vinder og beholder sin rank. Ved uafgjort bliver **første** rod barn af **anden** rod, og andens rank vokser med 1.
4. **Komprimér** undervejs: når **Find-Set** har nået roden, peger alle knuder på stien direkte på roden. Komprimering ændrer aldrig rank.
5. **Aflæs** de endelige forælderpegere. Roden, hvor $x.p = x$, er gruppens ID.

Make-Set (x) $x.p = x$ $x.rank = 0$	Find-Set (x) if $x \neq x.p$ $x.p = \text{Find-Set}(x.p)$ // path compression return $x.p$
Union (x, y) Link(Find-Set (x), Find-Set (y))	Link (x, y) // x, y er rødder if $x.rank > y.rank$ $y.p = x$ else $x.p = y$ if $x.rank == y.rank$ $y.rank = y.rank + 1$

Rank vs. højde

Rank er en øvre grænse for højden, ikke den faktiske højde. Komprimering flader træet ud uden at sænke rank, så rank kan ende større end den højde, du tegner. Hold kun styr på rank i rødderne.

Rækkefølge ved uafgjort

Rækkefølgen tæller ved uafgjort. $\text{Union}(x, y)$ med ens rank gør $\text{root}(x)$ til barn af $\text{root}(y)$, som vokser. Bytter du x og y , vender træet om.

Sti-komprimering

Komprimering kører kun på de Find-Set-kald, unionerne faktisk udfører. Komprimér ikke alle knuder til sidst. Et afsluttende Find-Set på en rod ændrer intet.

Priser

Tallene gælder for n Make-Set, $n - 1$ Union og m Find-Set.

Implementation	Find	Union	I alt
Lænket liste, naiv	$O(1)$	$O(n)$	$O(m + n^2)$
Lænket liste, vægtet union	$O(1)$	$O(n)$	$O(m + n \log n)$
Skov, ingen balancering	$O(n)$	$O(1)$	$O(m \cdot n)$
Skov, rank + komprimering	$\alpha(n)$	$\alpha(n)$	$O(m \cdot \alpha(n) + n)$

Vægtet union og inverse Ackermann

Vægtet union på länkede lister giver $O(n \log n)$: en knudes header omskrives kun, når dens liste er den korteste, så gruppen fordobles hver gang, altså højst $\log n$ omskrivninger pr. knude. For skoven er $\alpha(n)$ den inverse Ackermann-funktion, under 5 for ethvert tænkeligt n . Beviset (CLRS 19.4) er ikke pensum.

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 25

Disjunkt-mængde-skov med union by rank og path compression (Cormen 4. udg.). Fra tom: Make-Set på a, b, c, d, e, f , derefter $\text{Union}(f, e)$, $\text{Union}(b, f)$, $\text{Union}(d, a)$, $\text{Union}(f, d)$, $\text{Union}(b, c)$, $\text{Find-Set}(a)$. Hvilken figur viser strukturen bagefter?

- (a) Rod a med børnene b, c, d, e direkte under a ; f peger på e ($f \rightarrow e \rightarrow a$).
- (b) Rod a med b, c, d, e, f alle direkte under a (flad stjerne).
- (c) Rod a med børnene c, d, e ; under e ligger b og f ($b, f \rightarrow e \rightarrow a$).
- (d) Rod c øverst; $a \rightarrow c$; $b, d, e \rightarrow a$; $f \rightarrow e$.

Svar. Mulighed (a).

Fremgangsmåde.

- $\text{Union}(f, e)$: uafgjort $\rightarrow f.p = e$, $\text{rank}(e) = 1$.
- $\text{Union}(b, f)$: $\text{find}(f) = e$; $\text{rank}(e) > \text{rank}(b) \rightarrow b.p = e$.
- $\text{Union}(d, a)$: uafgjort $\rightarrow d.p = a$, $\text{rank}(a) = 1$.
- $\text{Union}(f, d)$: $\text{find}(f) = e$, $\text{find}(d) = a$, begge rank 1, uafgjort $\rightarrow e.p = a$, $\text{rank}(a) = 2$.
- $\text{Union}(b, c)$: $\text{find}(b)$ går $b \rightarrow e \rightarrow a$ og komprimerer b til a ; $\text{find}(c) = c$; $\text{rank}(a) > \text{rank}(c) \rightarrow c.p = a$.
- $\text{Find-Set}(a)$: a er rod, intet sker.

Endelige forældre: $b, c, d, e \rightarrow a$ og $f \rightarrow e$.

DM507 juni 2008, Opgave 3b/3c

Kør instruktionerne $\text{Union}(b, a)$, $\text{Union}(b, c)$, $\text{Union}(e, d)$, $\text{Union}(e, c)$, $\text{Union}(g, f)$, $\text{Union}(e, g)$ med union by rank. Ved uafgjort hænges $\text{root}(x)$ under $\text{root}(y)$, og $\text{root}(y)$'s rank forhøjes. Tegn skoven (b) uden path compression og (c) med.

Svar. Rod a (rank 2) med børnene b, c, d, f ; e under d ; g under f . Med komprimering flytter e direkte ind under a .

Fremgangsmåde. (b) uden komprimering:

- $\text{Union}(b, a)$: uafgjort $\rightarrow b.p = a$, $\text{rank}(a) = 1$.
- $\text{Union}(b, c)$: $\text{root}(b) = a$ (rank 1) $> c$ (rank 0) $\rightarrow c.p = a$.
- $\text{Union}(e, d)$: uafgjort $\rightarrow e.p = d$, $\text{rank}(d) = 1$.
- $\text{Union}(e, c)$: $\text{root}(e) = d$ (rank 1), $\text{root}(c) = a$ (rank 1), uafgjort $\rightarrow d.p = a$, $\text{rank}(a) = 2$.
- $\text{Union}(g, f)$: uafgjort $\rightarrow g.p = f$, $\text{rank}(f) = 1$.
- $\text{Union}(e, g)$: $\text{root}(e) = a$ (rank 2) $> \text{root}(g) = f$ (rank 1) $\rightarrow f.p = a$.

Resultat: a (rank 2) med børnene b, c, d, f ; e under d ; g under f .

(c) med komprimering: eneste find på en sti af længde 2 er trin 6, hvor $\text{find}(e)$ går $e \rightarrow d \rightarrow a$ og komprimerer e til at pege på a . Samme træ som (b), men e er nu direkte barn af a . Rank uændret.

DM02 jan 2006, Opg 3a

De første **5** Kruskal-kanter er valgt, så grupperne er $\{E, F, H, I\}$, $\{D, G\}$, $\{A, B\}$, $\{C\}$. Hvilken kant tilføjer Kruskal som den **6.**, og hvordan ser disjunkt-mængde-skoven ud bagefter, med union by rank + path compression?

Svar. Næste kant er $D-H$ (vægt **4**). D hænges under E .

Fremgangsmåde. Scan resterende kanter i stigende vægt. Den letteste med endepunkter i forskellige grupper er $D-H$: $D \in \{D, G\}$ og $H \in \{E, F, H, I\}$.

$\text{Union}(D, H)$ kalder Find-Set på begge:

- $\text{find}(D) = D$ (rank 1).
- $\text{find}(H)$ går $H \rightarrow I \rightarrow E$, så $\text{find}(H) = E$ (rank 2). Komprimering peger nu H og I direkte på E .

Rank 1 < rank 2, så D (med barnet G) hænges under E , der forbliver rank 2.

Fælden: $\text{find}(H)$ komprimerer, så kanten $H \rightarrow I$ forsvinder og H, I peger direkte på E .

Kruskal

Kruskal er stedet, union-find oftest dukker op. Opskrift: sortér kanterne efter stigende vægt; for hver kant, foren endepunkterne hvis deres Find-Set er forskellige, ellers spring over. Stop ved $|V| - 1$ kanter. Antal komponenter undervejs er $|V|$ minus accepterede kanter.

Grafgennemløb og sammenhængskomponenter

Et grafgennemløb starter i én knude og udforsker grafen kant for kant. Hver knude farves hvid (ikke set), grå (set, ikke færdig) eller sort (færdig). Metoderne adskiller sig kun i, hvilken grå knude du arbejder videre på.

BFS bruger en kø (FIFO), breder sig ud i ringe og finder den korteste vej i antal kanter.

DFS bruger en stak eller rekursion, dykker så dybt den kan før den vender om, og giver hver knude tidsstempler for opdagelse og afslutning.

Til eksamen kører du typisk BFS eller DFS i hånden på en lille orienteret graf, klassificerer kanter, laver en topologisk sortering eller tæller sammenhængskomponenter. Begge kører i

$$O(n + m)$$

hvor n er antal knuder og m antal kanter.

Sådan løser du den

DFS i hånden

1. Sortér hver naboliste **alfabetisk** det gør rækkefølgen entydig.
2. Hold én global tæller `time = 0`. Start i den givne knude a .
3. Opdag en hvid knude (hvid $\rightarrow$ grå): `time += 1, u.d = time`. Gå ned i dens første ubrugte hvide nabo.
4. Ingen ubrugte hvide naboer tilbage: afslut knuden (grå $\rightarrow$ sort): `time += 1, u.f = time`, og gå tilbage.
5. Er der hvide knuder tilbage efter starttræet, starter det ydre loop i næste hvide knude.

Hver knude får et interval $[d, f]$. To tik per knude giver sidste afslutningstid

$$2n$$

Parentes-strukturen

Intervallerne nestes som parenteser: $[u.d, u.f]$ og $[v.d, v.f]$ er enten adskilte eller det ene ligger helt inde i det andet. En knude er grå præcis i sit eget interval.

Kantklassifikation (u, v), set fra u

1. v hvid $\rightarrow$ tree-kant. Her er $u.d < v.d < v.f < u.f$.
2. v grå $\rightarrow$ back-kant: v er en forfader, der stadig ligger på stakken. En sløjfe til knuden selv tæller med.
3. v sort og $u.d < v.d \rightarrow$ forward-kant: v er en allerede afsluttet efterkommer.
4. v sort og $u.d > v.d \rightarrow$ cross-kant: v ligger i et undertræ, der blev afsluttet før.

Uorienteret kanttype

En uorienteret graf har kun tree- og back-kanter. Forward- og cross-kanter kræver en orienteret graf. Nævner en MCQ “én kant af hver type”, er grafen orienteret.

Back-kant og kredse

En orienteret graf har en kreds netop hvis DFS finder en back-kant; ingen back-kanter betyder en DAG. Forveksl ikke back-kant med cross-kant.

BFS i hånden

1. Sortér nabolisterne **alfabetisk**. Sæt start a til $d = 0$ og læg den i køen.
2. Tag u ud forrest. For hver hvid nabo v i rækkefølge: $v.d = u.d + 1$, $v.\pi = u$, læg v bagest.
3. Køen holder altid de grå knuder, og d -værdierne falder aldrig.
4. Når køen er tom, er $v.d$ afstanden fra s til v i kanter, eller ∞ hvis v ikke kan nås:

$$v.d = \delta(s, v)$$

BFS niveaurækkefølge

Alle knuder i afstand i kommer ud før nogen i afstand $i + 1$. Vil du finde den første med $d = k$, fyld et niveau ad gangen: niveau 1 er de sorterede naboer til s , og udvid dem i rækkefølge.

Topologisk sortering og komponenter

1. Topologisk sortering (kun DAG): kør DFS og list knuderne efter faldende u.f. Så peger hver kant fra venstre mod højre. Tjek en foreslået rækkefølge ved at se, at hver kant $u \rightarrow v$ har u før v .
2. Sammenhængskomponenter (uorienteret): kør DFS eller BFS med et ydre loop. Hvert ydre kald fejer en hel komponent, så tæl kaldene.
3. Stærkt sammenhængende komponenter (orienteret, Kosaraju): kør DFS på G og gem afslutningstider. Vend alle kanter til G^T . Kør DFS på G^T med knuderne i faldende afslutningsrækkefølge; hvert træ er én SCC.

Tilbagevendende eksamensspørgsmål

MCQ juni 2023, spm. 14

Udfør $\text{BFS}(G, a)$ med start i knude a og nabolister sorteret alfabetisk. Orienterede kanter: $f \rightarrow g$, $g \rightarrow j$, $i \rightarrow j$, $c \rightarrow f$, $c \rightarrow g$, $d \rightarrow g$, $d \rightarrow i$, $h \rightarrow c$, $b \rightarrow c$, $e \rightarrow b$, $e \rightarrow j$, $a \rightarrow d$, $a \rightarrow e$. Hvilken knude er den første, der får tildelt d -værdi (afstand) 4?

- (a) c
- (b) f
- (c) h
- (d) i
- (e) j

Svar. (b) f .

Fremgangsmåde. Niveau for niveau fra a : $d = 1$: d, e . $d = 2$: g, i (fra d), b, j (fra e). $d = 3$: c (fra b). $d = 4$: f (fra c). Først til afstand 4 er f .

MCQ juni 2023, spm. 16

DFS-Visit(G, a) på den orienterede graf med kanter $a \rightarrow b, b \rightarrow c, c \rightarrow d, a \rightarrow e, b \rightarrow f, c \rightarrow f, c \rightarrow g, f \rightarrow a, f \rightarrow g, d \rightarrow h, h \rightarrow g$. Hvor mange **forward-kanter** er der i dette gennemløb?

- (a) 0
- (b) 1
- (c) 2
- (d) 3
- (e) 4

Svar. (c) 2.

Fremgangsmåde. Tider: $a(1/16), b(2/13), c(3/12), d(4/9), h(5/8), g(6/7), f(10/11), e(14/15)$. Back-kant: $f \rightarrow a$ (a grå). Cross-kant: $f \rightarrow g$ (g sort, $d[f] = 10 > d[g] = 6$). Forward-kanter: $c \rightarrow g$ ($d[c] = 3 < d[g] = 6$) og $b \rightarrow f$ ($d[b] = 2 < d[f] = 10$). I alt 2.

MCQ juni 2023, spm. 17

Orienteret graf med kanter $a \rightarrow b, a \rightarrow d, b \rightarrow c, b \rightarrow d, b \rightarrow e, c \rightarrow f, e \rightarrow f, g \rightarrow c, g \rightarrow f$. Hvilke lister er en topologisk sortering? (Et eller flere svar.)

- (a) a, b, c, d, e, f, g
- (b) a, b, e, d, c, g, f
- (c) a, b, e, d, g, c, f
- (d) a, b, g, c, e, d, f
- (e) g, a, b, e, c, f, d

Svar. (c), (d) og (e).

Fremgangsmåde. Hver kant $u \rightarrow v$ skal have u før v . Bindinger: g før c, f ; b før c, d, e ; c, e før f . (a) og (b) sætter g efter c og bryder $g \rightarrow c$. (c), (d), (e) overholder alle kanter.

MCQ juni 2015, spm. 10

Udfør DFS med start i i med nabolisterne sorteret alfabetisk. Hvilken knude får opdagelsestid (starttid) 12? Hjulgraf med center i og ydre knuder a til h . Eger: $i \rightarrow a, i \rightarrow b, c \rightarrow i, i \rightarrow d, e \rightarrow i, f \rightarrow i, i \rightarrow g, i \rightarrow h$. Ydre kanter: $h \rightarrow a, b \rightarrow a, b \rightarrow c, d \rightarrow c, e \rightarrow d, f \rightarrow e, g \rightarrow f, g \rightarrow h$.

- (a) Knuden d
- (b) Knuden e
- (c) Knuden f
- (d) Knuden g
- (e) Ingen knude har starttid 12

Svar. (b) knuden e .

Fremgangsmåde. Naboliste (sorteret): $i : [a, b, d, g, h]$, $b : [a, c]$, $c : [i]$, $d : [c]$, $e : [d, i]$, $f : [e, i]$, $g : [f, h]$, $h : [a]$. DFS fra i : i opd 1; a opd 2 afs 3; b opd 4; c opd 5 afs 6; b afs 7; d opd 8 afs 9; g opd 10; f opd 11; e opd 12 afs 13; f afs 14; h opd 15 afs 16; g afs 17; i afs 18. Opdagelsestid 12 er e .

Minimum udspændende træer

Givet en vægtet, sammenhængende, urettet graf er et minimum udspændende træ (MST) den billigste måde at forbinde alle knuderne: vælg kanter så alt hænger sammen og den samlede vægt er mindst mulig.

Et udspændende træ har altid én kant mindre end antal knuder.

$$|\text{kanter i MST}| = n - 1$$

Til eksamen kører du Kruskal eller Prim i hånden. Spørgsmålet er typisk “hvilken kant tilføjes sidst”, “hvilken kant forkastes først”, eller “hvor mange komponenter er der nu”.

Sådan løser du den

Begge algoritmer er grådige og bygger på samme cut-idé: den letteste kant der krydser et cut gennem grafen er altid sikker at vælge. De adskiller sig kun i, hvordan de vælger næste sikre kant.

Kruskal — sortér kanterne, tag den letteste der ikke laver en kreds

1. Giv hver knude sin egen komponent.
2. Sortér alle kanter efter vægt, letteste først.
3. Gennemløb kanterne. Tag kant (u, v) hvis u og v ligger i hver sin komponent, og slå komponenterne sammen. Ligger de allerede sammen, laver kanten en kreds: spring den over.
4. Stop når du har $n - 1$ kanter.

Prim — voks ét træ ud fra en startknude

1. Vælg en startknude r . Den besøgte mængde C er $\{r\}$.
2. Find den letteste kant der går fra C ud til en knude udenfor. Tag kanten — den er en MST-kant — og gem den som den nye knudes forælder $v.\pi$, før du lægger knuden ind i C .
3. Gentag til alle knuder er i C . MST’et er nu de $n - 1$ kanter du tog: kantmængden $\{(v, v.\pi)\}$ for hver knude på nær startknuden.

Aflæs kanterne

Det er **kanterne**, ikke knuderne, der er træet — alle knuder ender jo med at være med uanset hvad. I bogens notation gemmer hver knude v sin indkomne kant som forælderen $v.\pi$, og hele MST’et er $A = \{(v, v.\pi) \mid v \notin \{r\}\}$. For Kruskal er det endnu mere ligetil: træet er de kanter du accepterede undervejs (dem der ikke lavede en kreds). Begge ender med $n - 1$ kanter.

Extract-Min

Prim køres ofte med en min-prioritetskø, og så spørger eksamen til Extract-Min frem for til selve træet, men det er samme algoritme. Hver knude uden for træet bærer en nøgle, nemlig vægten af den letteste kant der forbinder den til det du har bygget indtil nu (uendelig, hvis ingen kant rører den endnu). Extract-Min er den køoperation der fjerner og returnerer knuden med den mindste nøgle; den knude lægges ind i træet, og bagefter opdateres naboernes nøgler. Startknuden har nøgle 0 og kommer derfor ud først. Rækkefølgen knuderne forlader køen i, er den samme som rækkefølgen

de føjes til MST'et. Så "hvilken knude udtages sidst med Extract-Min" er bare "hvilken knude føjes sidst til træet", løst nøjagtig som Prim-opskriften ovenfor.

Prim som UCS

Kender du UCS (uniform cost search), så er Prim stort set UCS uden et goal: du trækker den billigste knude ud af køen, lægger den ind, og bliver ved til hele grafen er med i stedet for at stoppe ved et mål. Den ene forskel ligger i nøglen — i Prim vægter du den enkelte kant ind til træet, ikke den samlede vej fra start. Netop dét gør resultatet til et MST og ikke et korteste-vej-træ.

Entydigt MST

Er alle kantvægte forskellige, er MST'et entydigt: Kruskal og Prim giver samme træ, selv om de tager kanterne i hver sin rækkefølge.

Forskellig rækkefølge

Kruskal ser på alle kanter i vægtrækkefølge. Prim ser kun på kanter der rører den knudemængde han har bygget. De tager derfor sjældent kanterne i samme rækkefølge.

Hvornår forkastes en kant

En kant forkastes netop når begge endepunkter allerede sidder i samme komponent — det ville give en kreds.

Begge algoritmer kører på en sammenhængende graf i:

$$O(m \log n)$$

For Kruskal dominerer sorteringen af kanterne. For Prim er det en min-prioritetskø: n udtræk af nærmeste knude og op til m nøgleopdateringer.

Hver tagen kant smelter to komponenter til én, så efter k kanter gælder:

$$\text{antal komponenter} = n - k$$

Tilbagevendende eksamensspørgsmål

MCQ juni 2025, Spm. 21

Brug Kruskals algoritme til at finde et MST (læs næste spørgsmål først). Knuder $a, b, c, e, f, g, h, i, j$. Kanter med vægte: $(a, c) = 2$, $(a, b) = 15$, $(b, e) = 5$, $(b, f) = 8$, $(c, e) = 20$, $(c, j) = 17$, $(e, h) = 11$, $(e, j) = 9$, $(f, h) = 6$, $(f, g) = 3$, $(g, i) = 19$, $(h, i) = 16$, $(h, j) = 21$, $(i, j) = 14$. Hvilken kant tilføjes sidst til MST'et?

- (a) (a, b)
- (b) (i, j)
- (c) (h, j)
- (d) (e, h)
- (e) (h, i)
- (f) (e, j)

Svar. (a) (a, b) .

Fremgangsmåde. 9 knuder, så 8 kanter. Letteste først: $ac(2)$, $fg(3)$, $be(5)$, $fh(6)$, $bf(8)$, $ej(9)$, spring $eh(11)$ over (kreds), $ij(14)$. Til sidst binder $ab(15)$ $\{a, c\}$ til resten — den 8. og sidste kant.

MCQ juni 2025, Spm. 22

Fortsæt med Kruskals algoritme på samme graf. I det første øjeblik hvor en undersøgt kant ikke tages med, hvor mange sammenhængskomponenter har (V, A) ? (V er alle knuder, A er de kanter der er taget indtil nu.)

- (a) 1
- (b) 2
- (c) 3
- (d) 4
- (e) 5
- (f) 6

Svar. (c) 3.

Fremgangsmåde. $n = 9$. Tagne kanter: $ac(2)$, $fg(3)$, $be(5)$, $fh(6)$, $bf(8)$, $ej(9)$ — 6 stk. Næste, $eh(11)$, er den første forkastede (e og h allerede forbundet via $e-b-f-h$). Med 6 kanter: $9 - 6 = 3$ komponenter.

MCQ juni 2023, Spm. 20

Brug Prims algoritme til at finde et MST med start i knude a . Urettede vægtede kanter: $bc = 5$, $ca = 11$, $af = 10$, $fe = 7$, $bd = 13$, $ch = 9$, $ah = 2$, $ai = 3$, $fi = 4$, $eg = 8$, $dh = 1$, $hi = 6$, $ig = 12$. Hvilken knude tilføjes sidst til MST'et?

- (a) b
- (b) d
- (c) e
- (d) g

Svar. (a) b .

Fremgangsmåde. Voks C fra a ; hvert skridt tager den letteste kant ud af C og trækker en ny knude ind. Den kant er en MST-kant:

Skridt	Knude ind	Kant taget
1	h	$ah = 2$
2	d	$hd = 1$
3	i	$ai = 3$
4	f	$fi = 4$
5	e	$fe = 7$
6	g	$eg = 8$
7	c	$ch = 9$

Skridt	Knude ind	Kant taget
8	b	$bc = 5$

De otte kanter i højre kolonne er MST'et (9 knuder giver 8 kanter). Sidste knude ind er b , så svaret er (a).

MCQ juni 2019, Spm. 17

Kør Kruskals algoritme på grafen G_3 . Knuder $a, b, c, d, e, f, g, h, i$ med vægtede kanter: $(a, f) = 3$, $(a, b) = 4$, $(f, g) = 8$, $(f, b) = 5$, $(g, b) = 2$, $(g, c) = 6$, $(b, c) = 1$, $(h, c) = 9$, $(h, d) = 6$, $(h, i) = 9$, $(i, d) = 7$, $(i, e) = 1$, $(c, d) = 8$, $(d, e) = 7$. Hvilken kant er den første der undersøges af algoritmen, men ikke tages med i MST'et?

- (a) Kanten (b, f)
- (b) Kanten (c, g)
- (c) Kanten (d, i)
- (d) Kanten (d, e)
- (e) Kanten (f, g)
- (f) Kanten (h, i)

Svar. (a) kanten (b, f) , vægt 5.

Fremgangsmåde. Letteste først: $bc(1)$, $ie(1)$, $gb(2)$, $af(3)$, $ab(4)$ — nu er $\{a, b, c, f, g\}$ én komponent. Næste, $(f, b) = 5$, har begge endepunkter i den komponent: en kreds, og den første forkastede kant.

Korteste veje

Du har en orienteret, vægtet graf og en startknode s . Du vil finde vejen fra s til hver anden knude hvor vægtene summer til mindst muligt. Den letteste vægt til v kalder vi $\delta(s, v)$.

$$\delta(s, v) = \text{vægten af den letteste vej fra } s \text{ til } v$$

Alle algoritmerne bygger på samme skridt: **relax**. Du gættet en afstand til hver knude og forbedrer gættet når du finder en kortere vej. Til eksamen skal du vælge den rigtige algoritme til en graf, køre en i hånden, eller sammenligne køretider for alle-par-afstande.

Sådan løser du den

To operationer bærer det hele. Sæt alle gæt til uendeligt og startknudens til nul. Slap så kanter af, én ad gangen.

Relax — det fælles skridt

1. Initialiser: $v.d = \infty$ og $v.\pi = \text{NIL}$ for alle knuder, derefter $s.d = 0$. Her er $v.d$ dit bedste afstandsgæt og $v.\pi$ forgængerens på vejen.
2. Relax kanten (u, v) : er vejen gennem u kortere end gættet, så opdatér.
3. Altså hvis:

$$v.d > u.d + w(u, v)$$

så sætter du

$$v.d \leftarrow u.d + w(u, v), \quad v.\pi \leftarrow u$$

Relax-egenskaber

Uanset hvilken relax-algoritme du kører: gættet er aldrig for lavt ($\delta(s, v) \leq v.d$), det kan kun falde, og når $v.d$ rammer $\delta(s, v)$ ændres det aldrig igen.

Algoritmerne adskiller sig kun ved rækkefølgen kanterne slappes af i, og den afhænger af grafen. Læs derfor grafen først.

Vælg algoritmen

1. Vægtene: alle ens (enhedsvægte)? Alle ikke-negative? Eller nogle negative?
2. Strukturen: er grafen en DAG (orienteret uden kredse), eller har den kredse?
3. Match mod forudsætningerne:
 - **BFS** virker kun hvis alle vægte er ens.
 - **Dijkstra** kræver at alle vægte er ≥ 0 .
 - **DAG-Shortest-Paths** kræver at grafen er en DAG. Vægtene må gerne være negative.
 - **Bellman-Ford** virker på enhver én-kilde-graf uden negativ kreds; negative vægte er fine.
 - **Floyd-Warshall** er til alle-par, tåler negative vægte, men ingen negativ kreds.
4. Vælg hver algoritme hvis forudsætning er opfyldt. Til “et eller flere svar” kan flere være rigtige.

MST er ikke korteste vej

Kruskal og Prim bygger et minimalt udspændende træ, ikke korteste veje. DFS alene gør det heller ikke. Klassiske distraktorer.

Forudsætninger pr. graf

En DAG med negative vægte: Dijkstra ude, Bellman-Ford og DAG-Shortest-Paths ok. Ikke-negative vægte med kredse: Dijkstra, Bellman-Ford og (ved enhedsvægte) BFS ok, men ikke DAG-Shortest-Paths.

Dijkstra og Bellman-Ford skal du oftest køre i hånden. Dijkstra tager altid den nærmeste uafklarede knude først.

```

DIJKSTRA(G, w, s)           // alle vægte >= 0
  INIT-SINGLE-SOURCE(G, s); S = {}; Q = G.V
  while Q != {}
    u = EXTRACT-MIN(Q); S = S ∪ {u}
    for each v in G.Adj[u]: RELAX(u, v, w)

BELLMAN-FORD(G, w, s)       // negative vægte ok, opdager neg. kreds
  INIT-SINGLE-SOURCE(G, s)
  for i = 1 to |V|-1: for each edge (u,v): RELAX(u, v, w)
  for each edge (u,v): if v.d > u.d + w(u,v): return FALSE
  return TRUE

```

Kør Dijkstra i hånden

1. Start med $s.d = 0$, resten ∞ .
2. Træk gentagne gange knuden med mindst d ud af køen; ved uafgjort den alfabetisk mindste.
3. Slap dens udkanter af og notér udtrækningsrækkefølgen. De færdige afstande kommer i stigende orden.

Bellman-Ford slapper alle kanter af $|V| - 1$ gange og tjekker så en sidste gang om en kant stadig kan forbedres. Kan den det, findes en negativ kreds.

Køretider

Dijkstra med binært heap: $O((n + m) \log n)$. Bellman-Ford: $O(nm)$.

Til alle-par kører du enten en én-kilde-algoritme fra hver knude eller Floyd-Warshall direkte. Vinderen afhænger af grafens tæthed.

Floyd-Warshall: $O(n^3)$

Tæt graf ($m = \Theta(n^2)$): n kørsler af Dijkstra giver $O(n^3 \log n)$, så Floyd-Warshall vinder. Tynd graf ($m = \Theta(n)$): n kørsler af Bellman-Ford giver $\Theta(n^3)$, det samme som Floyd-Warshall.

Tilbagevendende eksamensspørgsmål

Which of the following algorithms can be used to find shortest paths on this graph? (Et eller flere svar.) G_1 er orienteret med knuder a, b, c, d, e og **alle vægte 1**, og grafen **har kredse**: $b \rightarrow a$, $a \rightarrow d$, $a \rightarrow e$, $b \rightarrow e$, $e \rightarrow d$, $c \rightarrow b$, $c \rightarrow e$, $c \rightarrow d$.

- (a) Dijkstra
- (b) Bellman-Ford
- (c) DAG-Shortest-Paths
- (d) Breadth-First-Search (BFS)
- (e) Depth-First-Search (DFS)

Svar. (a), (b) og (d): Dijkstra, Bellman-Ford og BFS.

Fremgangsmåde. Vægtene er alle 1, altså ikke-negative og ens. Dijkstra kræver kun ikke-negative vægte: ok. Bellman-Ford kræver kun ingen negativ kreds: ok. BFS virker fordi vægtene er ens. Grafen har kredse, så DAG-Shortest-Paths fejler. DFS er ikke en korteste-vej-algoritme.

MCQ juni 2015, Spm. 14

Which of the following algorithms can be used to find shortest paths on this graph? (Et eller flere svar.) G_2 er orienteret med knuder a, b, c, d, e , **nogle vægte -1** , og grafen er en **DAG** (topologisk orden c, b, a, e, d): $b \rightarrow a(1)$, $a \rightarrow d(-1)$, $a \rightarrow e(1)$, $b \rightarrow e(-1)$, $e \rightarrow d(1)$, $c \rightarrow b(1)$, $c \rightarrow e(-1)$, $c \rightarrow d(1)$.

- (a) Dijkstra
- (b) Bellman-Ford
- (c) DAG-Shortest-Paths
- (d) Breadth-First-Search (BFS)
- (e) Depth-First-Search (DFS)

Svar. (b) og (c): Bellman-Ford og DAG-Shortest-Paths.

Fremgangsmåde. Negative vægte, så Dijkstra er ude. Den topologiske orden c, b, a, e, d har ingen bagudkant: grafen er en DAG, og DAG-Shortest-Paths virker. Bellman-Ford tåler negative vægte uden negativ kreds: ok. BFS kræver ens vægte; DFS finder ikke korteste veje.

MCQ juni 2017, Spm. 15

Run Dijkstra's algorithm on graph G_2 below, starting at node a . The first node extracted from the priority queue (via EXTRACT-MIN) during the run is node a . Which node is the **sixth** one extracted? (Ved uafgjort: alfabetisk mindste navn.) Kanter: $a \rightarrow b(1)$, $a \rightarrow d(3)$, $d \rightarrow g(1)$, $d \rightarrow e(3)$, $g \rightarrow h(3)$, $g \rightarrow e(1)$, $h \rightarrow i(1)$, $b \rightarrow d(1)$, $b \rightarrow e(3)$, $b \rightarrow c(2)$, $e \rightarrow h(1)$, $e \rightarrow f(2)$, $c \rightarrow e(1)$, $c \rightarrow f(3)$, $f \rightarrow h(2)$, $f \rightarrow i(3)$.

- (a) Node d
- (b) Node e
- (c) Node f
- (d) Node g
- (e) Node h

Svar. (b): node e .

Fremgangsmåde. Udtrækningsrækkefølgen med færdige afstande: $a(0), b(1), d(2), c(3), g(3), e(4), h(5), f(6), i(6)$. a : relax $b = 1, d = 3$. b : relax $d \rightarrow 2, c \rightarrow 3, e \rightarrow 4$. d ($2 < 3$): relax $g \rightarrow 3$. Nu er $c = 3$ og $g = 3$ uafgjort, så alfabetisk c fjerde, g femte. Sjette er $e = 4$.

DM507 juni 2012, Opg. 4a

Kør Bellman-Ford fra a på den orienterede graf G_1 og angiv den endelige $v.d$ for alle knuder. Kanter: $a \rightarrow e(8), a \rightarrow f(10), a \rightarrow b(17), e \rightarrow h(-4), f \rightarrow h(-10), f \rightarrow g(25), g \rightarrow h(-12), g \rightarrow c(-3), b \rightarrow g(-5), c \rightarrow b(19), c \rightarrow d(2), d \rightarrow e(6), h \rightarrow d(1)$.

Svar.

v	a	b	c	d	e	f	g	h
$v.d$	0	17	9	1	7	10	12	0

Fremgangsmåde. Init $a.d = 0$, resten ∞ , relax alle kanter $|V| - 1 = 7$ gange. Vejene der realiserer afstandene: $a \rightarrow f \rightarrow h$ giver $h = 0$; $a \rightarrow b \rightarrow g$ giver $g = 12$; $g \rightarrow c$ giver $c = 9$; $h \rightarrow d$ giver $d = 1$; $d \rightarrow e$ giver $e = 7$. Ingen negativ kreds er nåelig fra a , så værdierne er endelige.

Korrekthed og løkkeinvarianter

En løkkeinvariant er et udsagn, der er sandt hver gang løkkebetingelsen testes. Holder det ved hver test, holder det også ved den sidste, hvor løkken stopper. Der ved du noget præcist om variablerne, og det giver svaret.

Beviset er induktion: invarianten holder før første test, ét gennemløb bevarer den, og løkken stopper til sidst. Til eksamen skal du enten **bevise** en given invariant eller **afgøre**, hvilke udsagn der er invarianter.

Sådan løser du den

Bevis en løkkeinvariant

1. Skriv invarianten op med kodens variabler, fx $s = i^2$.
2. **Initialisering.** Indsæt værdierne lige før første test, og tjek at udsagnet holder.
3. **Vedligeholdelse.** Antag det holder ved en test. Anvend kroppens tildelinger, og vis at det holder igen ved næste test.
4. **Terminering.** Vis at løkken stopper: en heltalsstørrelse aftager eller vokser strengt mod grænsen. Kombiner exit-betingelsen med invarianten (som stadig holder ved den fejlende test) og læs outputtet af.

Initialisering er basistilfældet, vedligeholdelse er induktionsskridtet, og terminering er hvor du høster korrektheden.

Afgør hvilke kandidater der er invarianter

1. Skriv starttilstanden op (værdierne lige før første test) og hvad kroppen gør ved hver variabel.
2. Tjek hver kandidat ved den **første** test. Er den falsk, kassér den.
3. Kør ét gennemløb i hånden og tjek igen. En kandidat, der bryder efter et skridt, ryger ud.
4. Behold udsagn, der er sande ved **hver** test, også exit-testen. Der kan være flere.

Alle tre dele

Vedligeholdelse alene beviser ikke korrekthed. Du skal bruge alle tre dele: rigtig start, hvert skridt bevarer det, og løkken stopper et sted, hvor invarianten giver svaret.

Exit-testen skyder forbi

Exit-testen kan skyde forbi grænsen. En `while x < n` kan stoppe med $x > n$, så $x \leq n$ er typisk **ikke** en invariant, selvom $x < n$ holder inde i kroppen. Tjek altid kandidaten ved den fejlende exit-test.

Sand ved første test

En invariant skal være sand **ved testen**, både første og sidste gang. $r = n!$ i en fakultetslønke er kun sandt ved sidste test, ikke første, så det er ikke en invariant, selvom det beskriver resultatet.

Tilbagevendende eksamensspørgsmål

DM507 juni 2009, Opg. 2

Betragt `KvadratRod(n)`:

```

i ← 0;  s ← 0
while s ≤ n:  s ← s + 2i + 1;  i ← i + 1
r ← i - 1;  return r

```

- (a) Bevis løkkeinvarianten: ved starten af while-løkken (altså ved løkketesten) gælder $s = i^2$.
 (b) Brug invarianten til at vise, at `KvadratRod` returnerer det største heltal $\leq \sqrt{n}$, altså $r = \lfloor \sqrt{n} \rfloor$.

Svar. Invarianten holder ved induktion. Ved exit er $r = i - 1 = \lfloor \sqrt{n} \rfloor$.

Fremgangsmåde. (a) Initialisering. Før første test er $i = 0$ og $s = 0$, så

$$s = 0 = 0^2 = i^2.$$

Vedligeholdelse. Antag $s = i^2$ ved en test. Kroppen sætter $s' = s + 2i + 1$ og $i' = i + 1$, så

$$s' = i^2 + 2i + 1 = (i + 1)^2 = (i')^2.$$

Altså holder $s = i^2$ ved hver test.

(b) **Terminering.** Løkken stopper første gang $s > n$. Da giver invarianten $i^2 = s > n$, så $i > \sqrt{n}$. Forrige test gik igennem, så $(i - 1)^2 \leq n$, altså $i - 1 \leq \sqrt{n}$. Dermed

$$i - 1 \leq \sqrt{n} < i \Rightarrow i - 1 = \lfloor \sqrt{n} \rfloor.$$

Returværdien er $r = i - 1 = \lfloor \sqrt{n} \rfloor$.

DM507 juni 2013, Opg. 6

Betragt `Factorial(n)`:

```

i ← n;  r ← 1
while i > 1:  r ← r · i;  i ← i - 1
return r

```

For heltal $n \geq 1$, angiv for hvert udsagn om det er en løkkeinvariant (sandt hver gang while-testen evalueres).

- (a) $i \geq 1$
- (b) $r = i!$
- (c) $r! \cdot i! = n!$
- (d) $r = n!/i!$
- (e) $r = n!$

Svar. Invarianter: (a) og (d).

Fremgangsmåde. Værdier ved testen: $(i, r) = (n, 1)$, så $(n-1, n)$, $(n-2, n(n-1))$, ..., ned til $(1, n!)$.

(a) $i \geq 1$: invariant. i løber $n, n-1, \dots, 1$, aldrig under 1.

(b) $r = i!$: nej. Ved første test er $r = 1$, men $i! = n!$.

(c) $r! \cdot i! = n!$: nej. Sand ved første test ($1! \cdot n! = n!$), men efter ét skridt er $(r, i) = (n, n-1)$, og $n! \cdot (n-1)! \neq n!$ for $n > 2$.

(d) $r = n!/i!$: invariant. Ved første test $n!/n! = 1 = r$. Kroppen sætter $r \leftarrow r \cdot i$ og dekrementerer i , hvilket bevarer det.

(e) $r = n!$: nej. Kun sand ved sidste test; $r = 1$ i starten.

DM507 juni 2012, Opg. 6

IntegerLog(n) beregner $\lfloor \log n \rfloor$:

```

    k ← 0;  i ← n
    while i > 1:  i lige : i ← i/2, k ← k + 1;  ellers : i ← i - 1
    return k

```

Brug fakta $\lfloor \log(n/2) \rfloor = \lfloor \log n \rfloor - 1$ og $\lfloor \log(n-1) \rfloor = \lfloor \log n \rfloor$ for ulige n .

(b) Bevis invarianten $\lfloor \log i \rfloor + k = \lfloor \log n \rfloor$.

(c) Vis, at løkken returnerer $\lfloor \log n \rfloor$.

Svar. Invarianten holder ved induktion. Ved exit er $i = 1$, så $k = \lfloor \log n \rfloor$.

Fremgangsmåde. (b) Initialisering. Før første test er $i = n$, $k = 0$, så

$$\lfloor \log i \rfloor + k = \lfloor \log n \rfloor + 0 = \lfloor \log n \rfloor.$$

Vedligeholdelse. Antag den holder ved en test med $i > 1$.

i lige: $i' = i/2$, $k' = k + 1$, så

$$\lfloor \log(i/2) \rfloor + (k + 1) = (\lfloor \log i \rfloor - 1) + k + 1 = \lfloor \log i \rfloor + k.$$

i ulige ($i \geq 3$): $i' = i - 1$, k uændret, og $\lfloor \log(i-1) \rfloor = \lfloor \log i \rfloor$.

(c) **Terminering.** Hvert skridt aftager det positive heltal i strengt, så løkken stopper ved $i = 1$. Her er $\lfloor \log 1 \rfloor = 0$, så invarianten giver $k = \lfloor \log n \rfloor$, som returneres.

MCQ juni 2017, Spm. 23

LogBaseTo(n) beregner $\lceil \log_2 n \rceil$:

```

    x ← 1;  r ← 0
    while x < n:  x ← 2x;  r ← r + 1
    return r

```

Hvilke af udsagnene nedenfor er en løkkeinvariant for algoritmen (altid sand når testen ved starten af while-løkken evalueres) for alle heltal $n \geq 1$? (Et eller flere svar.)

(a) $x = r + 1$

- (b) $2^r \cdot \log_2 n = \log_2(n/x)$
- (c) $x \leq n$
- (d) $\underline{2^r = x}$
- (e) $\underline{x < 2n}$

Svar. (d) og (e) er begge invarianter.

Fremgangsmåde. Tjek hvert udsagn ved hver test, exit-testen inklusive. Start: $x = 1$, $r = 0$; kroppen fordobler x og inkrementerer r .

(d) $2^r = x$: invariant. x starter som $2^0 = 1$, og hvert skridt fordobler x mens r vokser med 1.

(e) $x < 2n$: invariant. Inde i kroppen var $x < n$, så efter fordobling er $x < 2n$; det gælder også ved exit-testen.

(a) $x = r + 1$: nej. x vokser eksponentielt, r lineært.

(b) falsk ved simulation.

(c) $x \leq n$: nej. Exit-testen skyder forbi: $n = 3$ giver $x = 4 > 3$.

Appendiks

Appendix: Logaritmer og talrepræsentation

Et tal i base b er en vægtet sum af potenser af b . Cifret på plads i (talt fra 0 yderst til højre) vejer b^i . Decimal er base 10, binær base 2, hexadecimal base 16.

To slags spørgsmål går igen: logaritmeregler inde i O - og Θ -påstande (hvad vokser hurtigst), og omregning af heltal mellem baser i hånden. Begge er rutine, når du kan trinnene.

Sådan løser du det

En logaritme er det omvendte af en potens. $\log_2 x$ er det tal, du opløfter 2 i for at få x :

$$y = \log_2 x \iff 2^y = x$$

Her betyder $\log n$ altid $\log_2 n$. De tre regneregler:

$$\log(x_1 x_2) = \log x_1 + \log x_2$$

$$\log(x_1/x_2) = \log x_1 - \log x_2$$

$$\log(x^r) = r \log x$$

Basen er ligegyldig inde i O og Θ . Basisskift ganger blot med en konstant:

$$\log_a x = \frac{\log_b x}{\log_b a}$$

Så $\log_a n = \Theta(\log_b n)$ uanset basen. Derfor skriver man aldrig basen i en O -påstand.

Afgør om f er $O(g)$

1. Lær vækstrækken udenad; hver står stærkere end den foregående:

$$1 < \log n < (\log n)^k < n^\epsilon < n < n \log n < n^2 < n^3 < c^n < n^n$$

En polylog $(\log n)^k$ taber til enhver positiv potens n^ϵ . Konstante faktorer tæller ikke.

2. I tvivl? Regn grænseværdien

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

3. Aflæs svaret af L :

$$O : L < \infty \quad \Theta : 0 < L < \infty \quad o : L = 0 \quad \omega : L = \infty$$

4. Husk Stirling; den dukker op hvert år:

$$\log(n!) = \Theta(n \log n)$$

Omregn et positivt heltal til base b

1. Sæt $X = N$. Dividér X med $\underline{b}$ og notér **resten**; det er det næste ciffer, mindst betydende først.
2. Sæt X til **kvotienten** og gentag, så længe $X > 0$.
3. Resterne læst **nedefra og op** er cifrene i base $\underline{b}$.
4. Den anden vej, base b til decimal, er den vægtede sum

$$N = \sum_i d_i \cdot b^i$$

Hvert ciffer koster $O(1)$, og du deler N ned til 0, så omregningen tager $\Theta(\log_b N)$.

Hex som firkløvere

Hex er grupperet binær. Del bittene i firkløvere fra højre og erstat hver gruppe med ét hex-ciffer: 0–9, så $A = 10$ op til $F = 15$. F.eks. $0110\ 1010_2 = 6A_{16}$.

Toer-komplement

Toer-komplement. Øverste bit tæller negativt, $-(2^{k-1})$ i stedet for $+2^{k-1}$. Med 4 bit: $1101_2 = -8 + 4 + 0 + 1 = -3$. Et 1-tal forrest betyder altid et negativt tal.

bit	uden fortegn	toer-kompl.	bit	uden fortegn	toer-kompl.
0000	0	0	1000	8	−8
0001	1	1	1001	9	−7
0110	6	6	1110	14	−2
0111	7	7	1111	15	−1

Fortegnsskift

Fortegnsskift i toer-komplement er ikke “vend alle bit”. Kopiér bittene fra højre til og med første 1-tal, og vend resten. $0110 (= 6)$ bliver $1010 (= -6)$. (Samme som at vende alle bit og lægge 1 til.)

Vendte retninger

Vendte retninger er den klassiske fælde. n er **ikke** $O(\log n)$, og $\log n$ er **ikke** $\omega(n^2)$. $\log$ vokser langsommere end enhver positiv potens af n .

Tilbagevendende eksamensspørgsmål

MCQ juni 2021, Spm. 5

Hvilke udsagn er sande? $\log n$ er base to. (Et eller flere svar.)

- (a) n er $O(\log n)$
- (b) $(\log n)^3$ er $O(n^2)$
- (c) $n \log n$ er $O(n^{1.5})$
- (d) 2^n er $O(\sqrt{n})$
- (e) $3n^2$ er $O(n^2)$
- (f) 7^n er $O((\log n)^7)$
- (g) $\log(n!)$ er $O(n^2)$

Svar. (b), (c), (e) og (g) er sande.

Fremgangsmåde. Hold f/g op mod vækstrækken. (a) falsk, n slår $\log n$. (b) sand, polylog taber til ethvert polynomium. (c) sand, samme som $\log n = O(\sqrt{n})$. (d) falsk, eksponentiel slår enhver

potens. (e) sand, konstanten 3 forsvinder i O . (f) falsk, eksponentiel slår polylog. (g) sand, $\log(n!) = \Theta(n \log n)$, som ligger under n^2 .

MCQ juni 2021, Spm. 6

Hvilke udsagn er sande? $\log n$ er base to. (Et eller flere svar.)

- (a) $\log n$ er $\omega(n^2)$
- (b) $n^2 + n^3$ er $\Theta(n^3)$
- (c) 6 er $o(7)$
- (d) 3^n er $\Omega(2^n)$
- (e) $n/(\log n)^2$ er $o((\log n)^3)$
- (f) n^n er $\Omega(2^n)$
- (g) $n^{1.1}$ er $\omega(n \log n)$

Svar. (b), (d), (f) og (g) er sande.

Fremgangsmåde. Regn $L = \lim f/g$. (a) falsk, $L = 0$, så $o(n^2)$, ikke ω . (b) sand, n^3 dominerer, $L = 1$. (c) falsk, $6/7$ er konstant og $\neq 0$, så $\Theta(7)$, ikke o . (d) sand, $(3/2)^n \rightarrow \infty$. (e) falsk, $n/(\log n)^5 \rightarrow \infty$. (f) sand, $n^n/2^n \rightarrow \infty$. (g) sand, $n^{0.1}/\log n \rightarrow \infty$.

DM573, talReprSlides (omregningsøvelse)

Omregn $N = \underline{25}$ til binær.

Svar. $25 = 11001_2$.

Fremgangsmåde. Gentagen division med 2, rester læst nedefra og op:

division	kvotient	rest
25 : 2	12	1
12 : 2	6	0
6 : 2	3	0
3 : 2	1	1
1 : 2	0	1

Læst opad: 11001_2 . Tjek: $16 + 8 + 1 = 25$.

DM507 juni 2010, Opg. 3d

Algoritmen finder de binære cifre b_i for et positivt heltal n (her $n = \underline{55}$). Identiteten $x = y \cdot (x \text{ div } y) + (x \text{ mod } y)$ gælder for alle heltal. Hvad er køretiden?

```
BinaryDigits(n)
  i = 0
  d = n
  while d > 0
    b_i = d mod 2
```

```
d = d div 2
i = i + 1
k = i - 1
```

Svar. $\Theta(\log n)$.

Fremgangsmåde. d starter ved n og halveres (heltalsdivision) hver iteration, til den når 0. Antal iterationer er

$$\lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$$

Hver iteration laver $O(1)$ arbejde: én mod, én div, én addition. I alt $\Theta(\log n)$.